QuicCourier: Leveraging the Dynamics of QUIC-Based Website Browsing Behaviors Through Proxy for Covert Communication

Jianan Huang, Weiwei Liu, *Member, IEEE*, Guangjie Liu, *Member, IEEE*, Bo Gao, *Member, IEEE*, and Fengyuan Nie

***Abstract*—Network covert channels transmit secret messages by manipulating network traffic, including packet headers, timing intervals, and communication patterns. The growth of network services has spurred interest in exploring these channels. Yet, the practical application of these channels faces challenges in transmission rate and reliability due to unpredictable network interference. QUIC-based websites offer promising opportunities for covert communication, given their inherent dynamic nature from web resource updates and network interferences. Repeated visits or refresh actions on the same website generate substantial statistical redundancy. Furthermore, widely used proxy tools introduce additional traffic morphology changes. This paper presents *QuicCourier*, a covert channel leveraging web traffic's dynamic characteristics and proxy service encapsulation to hide messages in QUIC packets from the service node to the client. Guided by a generative model for web resource patterns, *QuicCourier* ensures that covert traffic closely resembles legitimate traffic, employing three packet-wise meta operations. The altered QUIC flows are then encased in proxy protocols, complicating the detection of embedded information. The covert receiver is incorporated into the proxy client. The efficacy of *QuicCourier* is evaluated using a dataset of over 30,000 web browsing traffic samples, demonstrating its exceptional undetectability against state-of-the-art traffic classification tools and a high covert transmission rate.**

***Index Terms*—Network covert channel, packet manipulation, proxy service, QUIC, Traffic analysis.**

I. Introduction

THE concept of a covert channel (CC) was first introduced in 1973 [1], describing a technique for transmitting secret messages between two parties without detection by a third party. In general, any communication data, resource states, or communication behaviors accessible to participants of a covert channel can serve as carriers. With the accelerating pace of the Internet, the prevalence of network packets has spurred the evolution of network covert channels. The effectiveness of a network covert channel is determined by three key performance metrics: undetectability, reliability, and covert transmission rate. To maintain resilience against a wide range of network traffic analysis tools and address inherent network interference, the covert transmission rate and reliability have emerged as primary constraints that impact the practical applicability of network covert channels.

Network covert channels are typically classified into two main categories: covert storage channels (CSCs), which use network protocol fields or payloads for covert communication, and covert timing channels (CTCs), which manipulate network packet timing information for covert purposes. However, CSCs are vulnerable to detection through traffic normalization and behavior analysis [2]. Moreover, the growing use of encryption protocols has constrained the evolution of CSCs, as more protocol fields are now encrypted. Furthermore, CTCs exhibit a heightened vulnerability to network latency and packet loss, which renders them challenging to operate effectively within real network environments. Due to the rigid structure of lower-layer protocols like IP, TCP, and UDP, as well as the variability of Internet jitter, an increasing number of upper-layer protocols are being studied for constructing network covert channels [3], [4], [5], [6].

In general, if any discernible element of network communication behavior can be manipulated to encode secret data while maintaining consistency with similar observable samples, a network covert channel can be established. In recent years, the construction of covert channels leveraging various network services has gained significant attention [4]. Common carriers include network multimedia and online games. For instance, a multimedia covert channel scheme proposed in [5] embeds secret messages by replacing encoded video frame data in video calling applications. However, this approach is vulnerable to man-in-the-middle attacks, as the transmission passes through uncontrolled application gateways. Similarly, for online games, decentralized game environments have been utilized to create covert channels between a user and the game server by replacing non-essential in-game messages [6]. Nonetheless, these methods

Received 7 March 2024; revised 1 March 2025; accepted 4 March 2025. Date of publication 7 March 2025; date of current version 4 September 2025. This work was supported in part by the National Natural Science Foundation of China under Grant U2436601, Grant U21B2003, Grant 62072250, and Grant 61602247, and in part by the National Key R&D Program of China under Grant 2021QY0700. *(Corresponding author: Weiwei Liu.)*

Jianan Huang, Weiwei Liu, Bo Gao, and Fengyuan Nie are with the School of Automation, Nanjing University of Science and Technology, Nanjing 210094, China (e-mail: jiananwong@njust.edu.cn; lwwnjust@njust.edu.cn; njustgb565@163.com; niefengyuan@njust.edu.cn).

Guangjie Liu is with the School of Electronics and Information Engineering, Nanjing University of Information Science and Technology, Nanjing 210044, China, and also with Key Laboratory of Intelligent Support Technology for Complex Environments, Ministry of Education, Nanjing 210044, China (e-mail: gjieliu@gmail.com).

Digital Object Identifier 10.1109/TDSC.2025.3549149

1545-5971 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.

are highly application-specific and heavily dependent on the state and behavior of the selected application.

It is well established that ideal carriers for covert channels in upper-layer network applications should be both ubiquitous and exhibit dynamic statistical characteristics. Among these, web browsing stands out as one of the most prevalent forms of network communication. The frequent updates to website content naturally introduce dynamic spatio-temporal traffic patterns, which not only complicate passive traffic analysis techniques, such as website fingerprinting, due to concept drift [7], [8], but also create significant opportunities for covert channels. Specifically, this inherent variability facilitates packet-level manipulations by leveraging the coarse-grained distribution of packet length-related features, enabling the embedding of secret data within web browsing traffic.

A key challenge in implementing such packet manipulation techniques lies in the complexity of modifying the underlying network stack to intercept traffic, embed secret data, and re-encapsulate it [9]. However, the increasing reliance on proxy services for web access offers a practical and efficient alternative [10]. Proxies are widely used for mitigating IP blacklisting, bypassing geo-restrictions, and automating web crawling, as well as for providing encryption to thwart deep packet inspection (DPI) [11]. Beyond these conventional applications, proxies can also be leveraged by individuals, including participants in covert communication, to actively engage in covert channels. Acting as intermediaries, proxies provide a natural interface for implementing packet manipulations during the encapsulation process, eliminating the need for modifications to the network stack.

Despite these advantages, proxy services still leak metadata, such as payload lengths and traffic directions, which can be exploited by wardens using statistical methods to infer user behaviors [12]. Therefore, for covert channels to achieve broad applicability, they must demonstrate resilience to scrutiny from unencrypted proxies and robustness against DPI. These requirements demand both behavioral consistency to preserve spatio-temporal characteristics and packet-level compliance with protocol specifications.

With the widespread adoption of HTTP/3 and its underlying QUIC protocol, QUIC traffic, with its inherent dynamic characteristics, has emerged as a suitable carrier for covert channels, offering several advantages for packet manipulation in the modulation of secret data. First, the packet header in QUIC traffic exposes only the minimal fields necessary for parsing, simplifying packet manipulation. This design enables modifications without requiring additional adjustments to exposed protocol fields, thereby reducing the complexity of covert packet manipulation. Second, QUIC's architecture, which operates over UDP, ensures that each packet is self-contained. This allows individual packets to be manipulated without causing anomalies or inconsistencies that might compromise the integrity of the session. Furthermore, QUIC's acknowledgment mechanism, which incorporates redundant packets, provides an efficient feedback channel. This mechanism can be leveraged to establish reliable covert communication by offering a robust method to confirm the receipt and integrity of secret data. Additionally, the QUIC protocol incorporates extensive state management within its encrypted messages, further enriching the dynamic statistical patterns inherent in legitimate traffic. As a result, the integration of the QUIC protocol with proxy services holds significant potential for enabling high-capacity, secure, and reliable covert channels.

In this paper, we present *QuicCourier*, an innovative proxy-based covert channel that leverages the dynamic spatio-temporal characteristics of QUIC-web browsing traffic, combined with the encapsulation capabilities provided by commonly used web proxy services. It facilitates the covert transmission of secret messages from the proxy node to the client through the manipulation of QUIC data within the proxy packets. This is achieved by employing a packet manipulation method guided by the characterization generative model for web resource pattern. The implemented meta-operations at the packet level encompass packet padding, packet insertion, and packet replacement, thereby preserving the inherent characteristics of web browsing traffic. In addition, we utilize the ACK packets in QUIC, which naturally include redundant frames, to address potential packet loss issues. These ACK packets provide feedback from the client to the proxy node, enhancing the reliability of the covert transmission.

The main contributions of this study can be outlined as follows:

- A novel framework for constructing network covert channels by combining packet manipulation and network proxy services is presented in this study. This framework leverages the prevalent behavior of accessing websites through proxies and capitalizes on the dynamic spatio-temporal characteristics of browsing traffic for the same website. Additionally, the encapsulation capabilities of web proxy services naturally facilitate inherent packet-wise operations. This framework provides a means to integrate various web services with web proxies, facilitating high-capacity covert communications in actual networks.
- A method for QUIC packet manipulation, guided by a characterization generative model for web resource pattern, has been developed within the proposed covert channel framework. This method encompasses two algorithms: **Web** **S**ignal Se**p**aration **a**nd **Re**combination (*WebSpare*) and **P**ayload **Le**ngth **S**election (*PLeS*). The *WebSpare* algorithm generates website resource patterns represented by run-length sequences of maximum payload size (MPS) packets. Concurrently, the *PLeS* algorithm ascertains the optimal length for packet insertion, crucial for maintaining the spatio-temporal characteristics of the traffic. This innovative method facilitates highly secure and reliable transmission of secret messages within web-browsing behavior.
- In evaluating the proposed covert channel scheme, both general and capability-maximizing adversarial models have been employed, utilizing a real-world dataset that includes over 30,000 samples of website browsing traffic. The general model emulates standard detection capabilities, whereas the capability-maximizing model exemplifies the most advanced detection potential. Notably, under the capability-maximizing adversarial model, even when

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SUMMARY OF EXISTING NETWORK COVERT CHANNEL SCHEMES

Dependency on Network Service	Carrier Object	Method Name	Type of Modulation Technique	Overview of Modulation Method
Service -independent	Packet fields	TCP/IP-CSC [15]	Packet bits modification	Encoding into unused or reserved fields
		IPv6-CSC [16]		
		TLS-CSC [17]		
		DYST [18]	Packet hash matching	Sending a signaling when the hash of a legitimate packet matches
	Inter-packet delays	IP-CTC [19]	Packet timing modification	Encoding into inter-packet delays
	Packet numbers	Cloak [20]	Packet dispatching	Encoding into the number of TCP packets over multiple flow
		LiHB [21]		Encoding into the number of HTTP requests over multiple flow
		RTP-CTC [22]	Packet dropping	Encoding into the number of RTP packets to be dropped
	Packet payload	Balboa [23]	Packet re-encryption	Replacing TLS payload with a pointer and secret data
	Packet format	FTE-CC [24]	Packet format transformation	Mimicking the unblocked protocol syntax
Service -dependent	Multimedia frames	DeltaShaper [25]	Video emulation	Feeding to the input of video stream
		Facet [26]		
		CoverCast [27]		
		Protozoa [5]	Data hooking	Replacing encoded video frames
		Stegozoa [28]		Video frame steganography
	In-game actions	Castle [29]	Automated game commands	Encoding into player actions
		Rook [30]	Game message replacement	Replacing mutable player actions
		Telepath [6]		Replacing non-disruptive game messages
	Transferred messages/files	SWEET [31]	Service tunneling	Sending as messages via email
		Stealth [32]		Sending as file via WebDAV and Git
		Dropbox-CC [4]		Encoding into file name and metadata in cloud storage
		Camoufler [33]		Sending as messages via instant messaging application
	Quic traffic	*QuicCourier*	Packet-wise operations	Embedding into dynamic spatio-temporal statistical redundancy of web browsing traffic

employing the advanced detection tool NeuTic, the detection accuracy for the proposed covert channel scheme registers at only about 60%, while achieving an average covert transmission rate of 4 bytes per packet.

The remainder of this paper is organized as follows. In Section II, we briefly summarize the related methods concerning covert channels and traffic identification. Section III details the adversary model and introduces the framework of *QuicCourier*. Section IV describes the packet manipulation method employed in *QuicCourier*. Section V discusses the experimental results and Section VI concludes this paper.

II. RELATED WORK

In this section, we provide an overview of network covert channels, focusing on service-independent and service-dependent types based on their reliance on specific network services. Additionally, we discuss various traffic identification methods potentially detecting these network covert channels.

A. Network Covert Channels

Over the past few decades, significant advancements have been made in the field of network covert channels, exploiting both network packets [13] and commonly used upper-layer services to conceal secret data by leveraging the inherent redundancy in network traffic [14]. Table I provides a comprehensive summary of existing covert channel schemes, highlighting four key aspects: dependency on network service, carrier object, type of modulation technique, and overview of modulation method.

Service-independent covert channels primarily leverage redundancies in network protocols or packet timing characteristics to modulate secret information. A common strategy for covert channels is CSCs, which exploit unused or reserved fields in protocols like TCP and IP to embed secret messages [15]. IPv6 extended headers [16] and multiple fields in the TLS handshake [17] are also used to hide data. However, as TLS formal data transmission involves fewer plaintext headers, and more headers are encrypted in emerging protocols like QUIC [34], the effectiveness of these methods is diminishing. Wardens can also sanitize or modify plaintext headers, further reducing their utility. To address these issues, DYST [18] uses ARP or RTCP packets as signals when the hash of a legitimate packet matches a portion of the secret message. While this approach enhances security, the scheme must wait for appropriate legitimate packets to appear.

To overcome reliance on specific protocol types, CTCs manipulate packet timing to encode secret data. One method encodes messages through inter-packet delays, though this is sensitive to network jitter [19]. Cloak [20] mitigates this by dispatching packets across multiple flows in distinct patterns. Similarly, LiHB [21], is an HTTP-based covert channel that discreetly transmits secret data by dispatching the number of requests in flows, though its covert bandwidth is low. Another example, Li et al. [22] developed a covert channel for RTP-based video calls by encoding secret messages within the sequence numbers of intentionally dropped packets. Despite their resilience, these timing-based methods can create detectable traffic patterns, making them vulnerable to statistical and machine learning-based detection [35], [36].

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.

In another, Balboa [23] utilizes the extracted session key to decrypt TLS packets, replacing the payloads with a pointer and secret data. The receiver then extracts the data and uses the pointer to restore the original data before sending it to the application. However, the dynamic nature of web traffic necessitates frequent updates to the pointer [37]. Similarly, Oakley et al. [24] propose a format transformation encryption (FTE) to tunnel covert traffic through host protocols that are difficult to block. Nevertheless, it limits in the static protocol with fixed spatio-temporal pattern for practical application.

The diversity of network services has led to the emergence of service-dependent covert channels, where services like multimedia streaming, cloud storage, and online games are exploited for covert communication. Multimedia applications, characterized by their substantial redundancy space, have played a pivotal role in the network covert channels. Instances like DeltaShaper [25], Facet [26], and CovertCast [27] encode secret data into live stream, often overlook anomalies introduced in the encoding process, thereby potentially elevating the risk of detection [38]. To minimize these risks, Protozoa [5] uses a hook technique to replace portions of encoded multimedia data with secret messages during WebRTC encapsulation. Stegozoa [28] builds on this approach by embedding secret data into multimedia frames encoding to avoid WebRTC gateway censorship. Although hook-based methods provide high security and substantial capacity, their effectiveness depends heavily on the status of applications and devices. Moreover, their implementation often requires extensive permissions, limiting their practical applicability.

Online games have also been leveraged to establish network covert channels. Systems like Castle [29] and Rook [30] embed secret messages into in-game behaviors, but rely on player actions for encoding, which results in limited bandwidth. In contrast, Telepath [6] achieves higher bandwidth by replacing non-disruptive messages with secret data during synchronization actions. To summarize, game-based covert channels generally lack universality, being confined to their specific design for a particular application.

Additionally, covert channels have also been developed using other Internet application services. SWEET [31] serves as an email tunnel that encapsulates traffic within email messages sent via Gmail. Another covert channel, Stealth [32], utilizes WebDav and Git for secret data transmission. In [4], a covert channel is implemented within the cloud service DropBox, enabling the exchange of covert information through file and metadata modification. Additionally, Camoufler [33] uses platforms like WhatsApp and Telegram to route web traffic through text messages, benefiting from the platforms' low latency. Although these methods blend well with legitimate traffic, they are vulnerable to man-in-the-middle attacks and often degrade service quality due to the low real-time nature of the channels.

In summary, the rise of encrypted protocols and packet headers has reduced the effectiveness of CSCs. While CTCs offer more flexibility by manipulating flow temporal behavior, they struggle to preserve legitimate spatio-temporal traffic patterns. With the rapid evolution of network services, techniques such as QUICstep [39] leverage encrypted tunnels to obfuscate sensitive handshake information and employ QUIC's connection migration to maintain seamless communication in the presence of censorship. This illustrates how core features of modern network protocols can be strategically utilized to evade restrictions while preserving performance. Similarly, covert channel construction has increasingly explored leveraging specific network services to enhance stealth and reliability. However, ensuring that these covert channels remain indistinguishable from legitimate traffic remains a critical challenge, necessitating further innovation in utilizing common network services for secure and resilient covert communication.

B. Network Traffic Identification

Network traffic identification technique depends on fixed fields and deep-level traffic characteristics to ascertain its source or type, playing a vital role in detecting network covert channels. For web browsing traffic via proxy, the identification methods can be categorized into website fingerprinting techniques, which utilize inherent IP, domain, and other spatio-temporal characteristics presented in the traffic of different websites to identify their respective origin sites. However, with the widespread adoption of network proxies, content delivery networks, and encryption protocols, IP or domain-based traffic identification methods are increasingly becoming outdated [40]. Modern techniques for web browsing traffic identification primarily focus on extracting statistical features and applying classical machine learning strategies, alongside incorporating traffic representation in deep learning models. In [41], a multinomial Naive Bayes classifier is used to capitalize on the frequency distribution of packet sizes, offering robustness against minor website alterations. In [42], support vector machines (SVM) are utilized, extending the feature representation to include characteristics like total transmitted bytes and the ratio of downstream to upstream packets, thereby significantly enhancing the accuracy of traffic classification. In the realm of classical machine learning-based traffic identification algorithms, AppScanner (AppS) is a notable method [7]. It analyzes packet length sequences and utilizes a set of 54 statistical features derived from upstream, downstream, and complete traffic, achieving an identification accuracy of 96% across 110 applications.

In contrast to feature engineering methods combined with classical machine learning, network traffic identification techniques based on deep learning eliminate the need for manual feature design. These methods, utilizing deep learning, have shown superior performance in identifying traffic across various public datasets. For example, a study in [43] introduces FS-Net, an encrypted traffic classifier utilizing a double-layer bidirectional gate recurrent unit (GRU). FS-Net excels in extracting deep features from packet length sequences and functions as an end-to-end classifier within a comprehensive framework, showing outstanding performance on a real-world dataset encompassing 18 applications. Deep Fingerprinting (DF) [44], a convolutional neural network (CNN)-based model for website fingerprinting attacks against Tor, employs direction sequences as input and achieves a 98% accuracy rate in closed-world scenarios. Additionally, EBSNN [45], a long short-term memory

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.

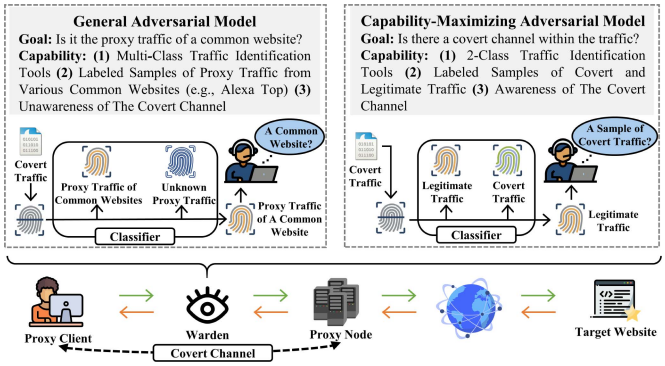


Fig. 1. General and capability-maximizing adversarial models for the proposed proxy-based covert channel framework.

(LSTM)-based model, is designed to handle unknown complex applications. It utilizes side-channel and payload features, employing hierarchical attention networks to learn high-level representations. In [46], an advanced deep learning approach named NeuTic is presented, specializing in the identification of TLS traffic with packet sequences as its primary input. This model integrates a multi-kernel convolution technique for processing variable-length features and a sequence self-attention mechanism, effectively capturing long-range dependencies in packet sequences.

III. System Design

In this section, we define the adversary model for our proxy-based covert channel framework, covering both typical and maximal capability scenarios. Then, we introduce the framework designed to establish a network covert channel, enabling the transmission of secret messages through proxy traffic by manipulating packets.

A. Adversary Model

Fig. 1 illustrates the adversarial scenario. Here, a user accesses a specific website through a proxy client, which collaborates with a proxy node to enhance Internet privacy. The proxy node serves as an intermediary, forwarding requests and retrieving website resource packets from the target website. This represents a standard privacy-enhancing web browsing activity via a proxy. However, within this seemingly normal activity, there exists a covert channel between the user and the proxy node. The proxy node, operated by a covert sender, discreetly transmits secret data to the user within the web browsing traffic via proxy.

The proposed covert channel scheme aims to mimic ordinary web browsing via proxy, ensuring undetectability, reliability, and practicality. For undetectability, the covert channel must avoid introducing abnormal network behavior or communication patterns, and the carrier communication should ideally be ubiquitous. Its traffic should closely resemble normal traffic across multiple dimensions of features, remaining indistinguishable even under advanced detection methods. For reliability, mitigating interference and packet loss is crucial, requiring a covert feedback mechanism to support acknowledgment and retransmission signaling, ensuring reliable communication under various network conditions. Lastly, for practicality, the design should minimize system modifications, utilize widely available network services, and ensure compatibility with existing devices, reducing deployment costs while maintaining normal service quality and availability.

For the purposes of our analysis, we assume that a warden can capture and analyze all network flows, including side-channel information such as packet payload, packet size, packet arrival time, and packet direction. In this scenario, covert transmissions are impermissible, whereas privacy-enhancing communication via proxy is viewed as a normal behavior. The warden possesses the capability to deduce the websites accessed by the proxy client via traffic identification tools.

In real-world scenarios, network traffic censorship typically begins with the detection of anomalies, followed by more detailed analyses. Reflecting this reality, we have developed two distinct adversarial models: the general and capability-maximizing models. These models are intended to simulate varying levels of capabilities and goals a warden may have in different operational environments.

In the general adversarial model, the warden's goal is to ascertain if the traffic originates from a proxy of a commonly accessed website. This aligns with the primary goals of most current web traffic monitoring systems. To achieve this goal, the necessary capabilities include having labeled samples of proxy traffic from a large number of common websites (e.g., websites listed in the Alexa Top Rankings), and the ability to train these labeled samples using traffic identification tools to construct a model that can associate proxy traffic with its target website. The warden remains oblivious to the specifics of the covert channel. The covert sender needs to ensure that the covert traffic is recognized by traffic identification tools as corresponding to the common target website of its carrier traffic, rather than being seen as an unknown proxy traffic sample, thereby preventing further analysis by the warden.

In contrast to typical capability scenarios within the general adversarial model, the capability-maximizing adversarial model empowers the warden with maximal capabilities. Specifically, the warden possesses a thorough comprehension of the covert channel method and precise labels for both legitimate and covert traffic samples. The warden's objective is to differentiate between legitimate and covert proxy traffic destined for a particular website. To accomplish this, a binary traffic identification tool is trained on these two sample categories to develop a decision model capable of detecting the existence of a covert channel. The covert channel is effective as long as the covert traffic remains indiscernible from legitimate traffic, even when the warden has access to labeled samples and advanced traffic identification tools. Therefore, covert traffic should exhibit similar characteristics to legitimate proxy traffic.

In addition, it is important to note that the warden employs passive techniques for network traffic analysis, refraining from active actions like packet delaying or dropping. Additionally, the decryption of the packets is not practically feasible. In general, the warden is assumed to access all network traffic

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.

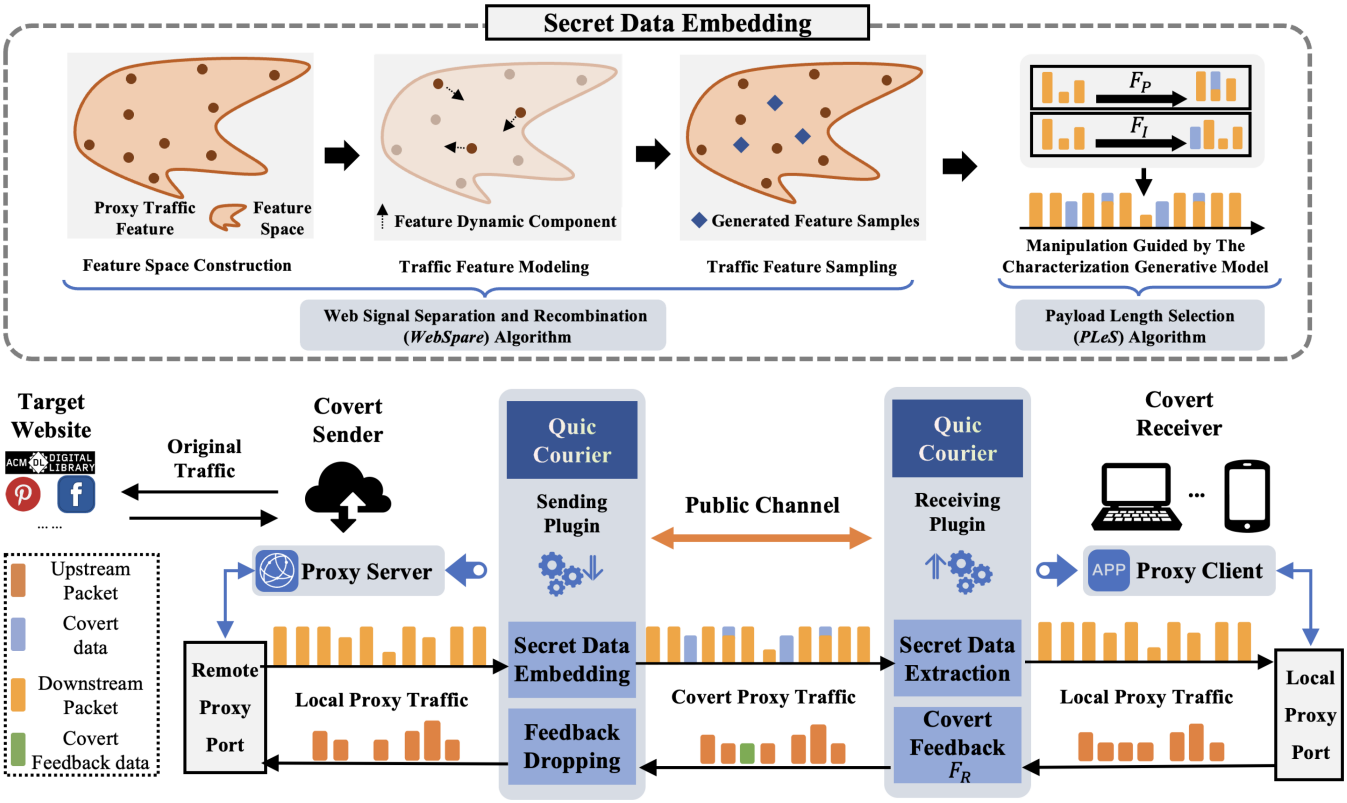


Fig. 2. An overview of the proposed *QuicCourier*.

but lacks the ability to access third-party devices within the network or decrypt traffic from third-party sources. This represents the typical maximum capability that a warden can attain in practice.

B. *Overview of* QuicCourier

The overview of *QuicCourier* is illustrated in Fig. 2. This framework serves as a network covert channel that can seamlessly integrate into a proxy system, such as Socks5, functioning as a pluggable transport layer. *QuicCourier* allows users to access websites through proxies while clandestinely receiving secret data from the proxy node. The dynamic spatio-temporal characteristics of web browsing traffic not only expand the feature space for traffic classifiers, making the detection of specific targets more complex but also create opportunities for establishing a network covert channel. Leveraging the inherent asymmetry of web browsing traffic, *QuicCourier* utilizes downstream traffic from the proxy node to the client to covertly embed secret data into the proxy traffic. This method of secret data embedding is achieved through packet manipulation, guided by a generative model characterizing web resource patterns. The packet manipulation method encompasses the *WebSpare* and *PLeS* algorithms. The *WebSpare* algorithm generates website resource patterns represented by run-length sequences of MPS packets. Concurrently, the *PLeS* algorithm determines the optimal length for packet insertion, which is crucial for preserving the spatio-temporal characteristics of the traffic.

In this scenario, the client functions in dual capacities: as both the proxy client and the covert receiver. Similarly, the proxy node acts as the proxy server and the covert sender. During website browsing by the client, the proxy node initiates requests for the desired web content and then relays the web resources to the client. For QUIC traffic proxying, a virtual network interface is essential. This interface sets up a logical gateway compatible with UDP packets, enabling all QUIC packets to be directed to this virtual gateway and subsequently rerouted to the local proxy port.

When packets return from the proxy node to the client, *QuicCourier* applies packet manipulations to embed secret data. To preserve the integrity of web content, *QuicCourier*'s proxy node utilizes packet padding F_P and packet insertion F_I for downstream packets, while packet replacement F_R is employed to incorporate feedback bytes into the upstream packets, thereby enhancing the reliability of the covert channel. To ensure broad applicability, these manipulations should resist protocol analysis even under unencrypted proxies. As the secret data is typically encrypted, either through pre-encryption or by the proxy's encryption layer, it remains indistinguishable from the encrypted payload. The primary focus is to ensure that the manipulated packets remain compliant with protocol specifications. Below, we define three fundamental packet manipulation techniques in QUIC that effectively preserve protocol conformity while embedding secret data.

Packet Padding $F_P(i, n)$: Padding the secret data of length n into the end of the i-th packet p_i. QUIC's encryption ensures

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.

that most of the packet, including the header, is protected, exposing only minimal fields like flags and connection IDs. As a result, packet padding does not require any header modifications, preserving the packet's integrity. In contrast, performing the same manipulation in popular TLS implementations requires adjusting the exposed length field in its header, which increases complexity and the risks of detection.

Packet Insertion $F_I(i, n)$: Inserting a covert packet of n-byte secret data payload with the packet header into the front of the i-th packet p_i. QUIC allows this inserted packet without disrupting the connection state due to its connectionless design and independent flow control management. In comparison, TLS, which operates over TCP, relies on maintaining a global connection state. TLS messages often span multiple packets, and inserting a packet requires to adjust the exposed headers of previous packets to maintain the correct connection state. This process is complex and error-prone, particularly if the packets have already been transmitted.

Packet Replacement $F_R(i, n)$: Replacing the payload of the i-th packet with secret data of the length n. QUIC incorporates ACK packets for packet loss recovery, and its acknowledgment mechanism includes cumulative acknowledgment information, which mitigates the impact of potential ACK packet loss. By exploiting this cumulative nature, ACK packets can sporadically be replaced with a small amount of feedback bytes to enhance the reliability of the covert channel. The proxy node subsequently extracts the feedback and intentionally drops the packet, simulating a scenario where the packet is lost due to network interference.

By utilizing the run-length sequence of MPS packets as the characterization of web resource patterns, we employ the *WebSpare* algorithm for traffic feature space construction, traffic feature modeling, and traffic feature sampling. The generated run-length sequence of MPS packets serves as the pattern that guides the packet padding and packet insertion operations. With this MPS-related pattern, each downstream packet corresponds to an expected payload length L_i.

Considering $n_{\max}$ to represent the maximum payload size of the UDP traffic, when the i-th packet's payload l_i is smaller than $n_{\max}$, and the expected payload length $L_i = n_{\max}$, packet padding F_P is employed to fill the packet with secret data up to the maximum payload size. Conversely, if the expected payload length $L_i < n_{\max}$, and the current packet payload length $l_i = n_{\max}$, packet insertion F_I is used to insert a covert packet before p_i with specific payload length determined by the *PLeS* algorithm. In cases where the expected payload length L_i equals the current packet payload length l_i, it indicates the consistent behavior of a resource or resource block transmission, negating the need for further packet manipulation. The packet operations corresponding to different $\{l_i, L_i\}$ are shown in Table II.

TABLE II
THE PACKET MANIPULATION FOR DIFFERENT $\{l_i, L_i\}$

l_i	L_i	Operation
$< n_{\max}$	$< n_{\max}$	/
$< n_{\max}$	$= n_{\max}$	F_P
$= n_{\max}$	$< n_{\max}$	F_I
$= n_{\max}$	$= n_{\max}$	/

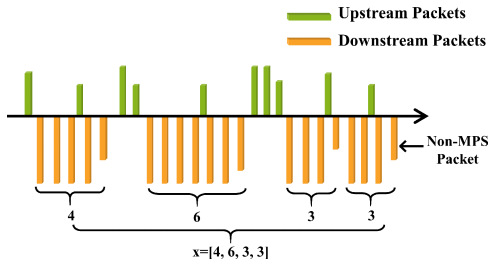


Fig. 3. Illustration of the run-length sequence of MPS packets.

IV. PACKET MANIPULATION METHOD IN *QUICCOURIER*

Section III provides an overview of *QuicCourier*, a covert channel framework that operates within a proxy system and utilizes QUIC web technology. This section introduces a packet manipulation method designed specifically for this framework. The method consists of two key algorithms: the Web Signal Separation and Recombination (*WebSpare*) algorithm and the Payload Length Selection (*PLeS*) algorithm. The *WebSpare* algorithm is responsible for generating a run-length sequence of maximum payload size (MPS) packets, effectively capturing the web resource transmission characteristics within the downstream traffic. On the other hand, the *PLeS* algorithm is tasked with identifying the optimal packet insertion length required by *QuicCourier* to maintain the spatio-temporal characteristics of the traffic.

A. WebSpare *Algorithm*

Within the downstream traffic of QUIC web browsing, a noteworthy characteristic is the intermittent appearance of non-MPS packets after several consecutive MPS packets. This phenomenon stems from variations in web resources and forms a vital distinguishing element in different web browsing traffic patterns. Consequently, it becomes imperative to characterize and maintain this type of feature as a prerequisite for the covert channel. In this study, we utilize a run-length sequence of MPS packets to depict this characteristic, as it can accurately represent the web resource pattern. It's worth noting that this resource-related feature typically exhibits stability over short time periods while displaying dynamic behavior over more extended time periods.

As illustrated in Fig. 3, to obtain the run-length sequence of MPS packets, the payload length sequence of downstream packets is initially partitioned based on the non-MPS packets, with each segment representing a resource or resource block in website browsing traffic. Given the downstream traffic containing T non-MPS packets, the run-length sequence of MPS packets can be represented by:

$$\mathbf{x} = [x_1, x_2, \ldots, x_T] \tag{1}$$

where x_i denotes the number of MPS packets between the $(i-1)$-th non-MPS packet and the i-th non-MPS packet. The run-length sequence $\mathbf{x}$ of MPS packets, hereinafter also referred to as the MPS-related pattern for convenience.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.

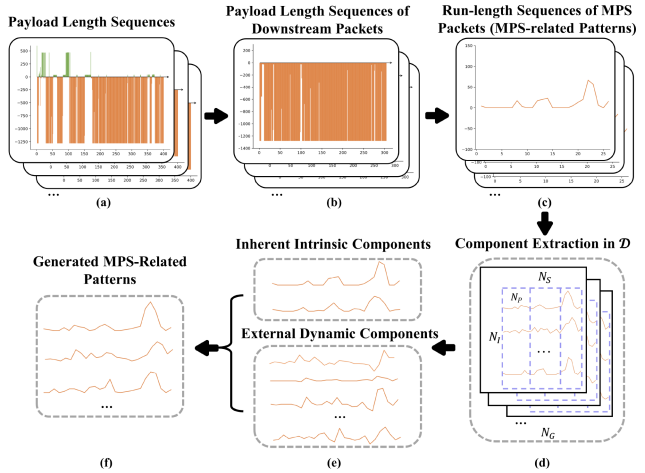


Fig. 4. Illustration of the *WebSpare* algorithm.

In the *WebSpare* algorithm depicted in Fig. 4, run-length sequences from historical traffic samples of a specific QUIC website are used to build a generative model of web resource patterns. Fig. 4(a)-(c) show these sequences of MPS packets (or MPS-related patterns), denoted as $\mathcal{X}$, extracted from payload length sequences of downstream packets from the same website, denoted as $\mathcal{S}$. Each sequence is a discrete time series where each value corresponds to a run-length of MPS packets. These sequences, although originating from the same site, are influenced by dynamic factors such as network interference. They comprise both inherent intrinsic component $\mathbf{b}=[b_1,\ldots,b_T]$ and external dynamic component $\mathbf{e}=[e_1,\ldots,e_T]$, expressed as follows:

$$\mathbf{x} = a_1\mathbf{b} + a_2\mathbf{e} \tag{2}$$

where a_1 and a_2 denote the corresponding weights in the MPS-related pattern $\mathbf{x}$, respectively. After determining $\mathbf{b}$ and $\mathbf{e}$, the subsequent step involves generating the MPS-related pattern $\mathbf{y}_{\text{gen}}$, which can be achieved through a summation of the two components. However, variability in a_1 and a_2 across sequences poses a challenge to the consistency of the generated patterns, and fixed weights can result in poor diversity. To address this issue, the proposed *WebSpare* algorithm incorporates a grouping strategy that maximizes reproducibility among groups, as depicted in Fig. 4(d)-(e).

To effectively implement the *WebSpare* algorithm, it is crucial to initially define the global parameters (N_I, N_P), which influence grouping and partitioning. The parameter N_I indicates the number of sequences in each group. Typically, the run-length sequences within these groups share similar collection times or originate from the same devices. On the other hand, the parameter N_P represents the size of each segment within run-length sequences. The segment is defined by a consecutive series of MPS packets and a non-MPS packet.

Upon completing the grouping and partitioning, MPS-related patterns extracted from the traffic samples of the same website are represented as a tensor $\boldsymbol{\mathcal{D}} = (\mathcal{D})_{g,i,s,p} \in \mathbb{R}^{N_G\times N_I\times N_S\times N_P}$. In this tensor, $g\in\{1,\ldots,N_G\}$ represents the group index, and N_G indicates the total number of groups. Correspondingly, $i\in\{1,\ldots,N_I\}$ signifies the index within a group, $s\in\{1,\ldots,N_S\}$ refers to the segment index within a sequence, while N_S denotes the total number of segments per sequence. Additionally, $p\in\{1,\ldots,N_P\}$ specifies the position within each segment. Furthermore, $\mathcal{D}_{g,i,s,:}$ is defined as the s-th segment of the i-th run-length sequence in the g-th group for convenience. In subscript notation, a colon in a dimension signifies selecting all elements in that dimension.

For each group $\mathcal{D}_g$, the generated MPS-related pattern $\mathbf{y}_{\text{gen}}$, which depicts the web resource transmission characteristics of the target website, can be defined as:

$$\mathbf{y}_{\text{gen}} = \sum_{s=1}^{N_S}\sum_{i=1}^{N_I} w_{i,s}\mathcal{D}_{g,i,s,:} = \sum_{s=1}^{N_S}\sum_{i=1}^{N_I}(w_{i,s}a_{1,i,s}\mathbf{b}_s + w_{i,s}a_{2,i,s}\mathbf{e}_s) \tag{3}$$

In the s-th segment, $w_{i,s}$ denotes the weight corresponding to the i-th sequence within this segment. The parameters $a_{1,i,s}$ and $a_{2,i,s}$ represent the intrinsic and dynamic component weights in the s-th segment of the i-th sequence, respectively. The intrinsic component in the s-th segment is represented by $\mathbf{b}_s$, whereas $\mathbf{e}_s$ is the dynamic component in the same segment. When the conditions $\sum_{i=1}^{N_I} w_{i,s}a_{1,i,s}=1$ and $\sum_{i=1}^{N_I} w_{i,s}a_{2,i,s}=0$ are considered in each segment, it enables the extraction of the intrinsic component. Similarly, the dynamic component is extracted using the same approach. Generally, direct extraction of these components presents challenges. Instead, inter-covariance is employed to approximate them and generate MPS-related patterns for each segment.

To efficiently extract distinct components and characterize the correlation relationships between $\mathcal{D}_{g_1}$ and $\mathcal{D}_{g_2}$, the inter-group covariance C_{g_1,g_2} is utilized. This method involves summing covariances across all possible group combinations, revealing behavioral dependencies and similarities in resource transmission among various groups. The inter-group covariance can be calculated as follows:

$$\sum_{g_1\neq g_2} C_{g_1,g_2} = \sum_{g_1\neq g_2}\sum_{i_1,i_2} w_{i_1}w_{i_2}\text{Cov}\left(\mathbf{v}_{g_1,i_1},\mathbf{v}_{g_2,i_2}\right) = \mathbf{w}^T\mathbf{U}\mathbf{w} \tag{4}$$

In this formulation, the indices g_1, g_2 range over the set $\{1,...,N_G\}$, while i_1, i_2 span $\{1,...,N_I\}$. Here, $\mathbf{v}_{g_1,i_1}$ denotes the i_1-th sequence within the g_1-th group, represented as $[\mathcal{D}_{(g_1,i_1,1,:)},\ldots,\mathcal{D}_{(g_1,i_1,N_S,:)}]$, and analogously for $\mathbf{v}_{g_2,i_2}$. The matrix $\mathbf{U}=(u_{i_1,i_2})^{N_I\times N_I}$ encapsulates the covariance between the i_1-th and i_2-th sequences across all distinct groups, where $u_{i_1,i_2}=\sum_{g_1\neq g_2}\text{Cov}(\mathbf{v}_{g_1,i_1},\mathbf{v}_{g_2,i_2})$. w_{i_1} and w_{i_2} represent the weights of the i_1-th and i_2-th sequence, respectively. The vector $\mathbf{w}=[w_1,\ldots,w_{N_I}]$ encompasses the weights for all sequences, with each element w_i corresponding to the weight of the i-th sequence.

To characterize the correlation relationships among different sequences within each group, along with their weight limitations, a normalization constraint is imposed. This normalization is crucial for bounding the solution space and preventing overfitting, and is mathematically defined as:

$$\sum_{i_1,i_2} w_{i_1}w_{i_2}\text{Cov}\left(\mathbf{r}_{i_1},\mathbf{r}_{i_2}\right) = \mathbf{w}^T\mathbf{Q}\mathbf{w} = 1 \tag{5}$$

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.

Considering the indices i_1 and i_2 within the set $\{1,\ldots,N_I\}$, $\mathbf{r}_{i_1}$ and $\mathbf{r}_{i_2}$ correspond to the i_1-th and i_2-th concatenated sequence from all groups, respectively. $\mathbf{r}_{i_1}$ is represented by $[\mathcal{D}_{(1,i_1,1,:)},\ldots,\mathcal{D}_{(1,i_1,N_S,:)},\ldots,\mathcal{D}_{(N_G,i_1,1,:)},\ldots,\mathcal{D}_{(N_G,i_1,N_S,:)}]$. Similarly, $\mathbf{r}_{i_2}$ is defined in the same way. The matrix $\mathbf{Q}=(q_{i_1,i_2})^{N_I\times N_I}$ captures the covariances between the concatenated sequences $\mathbf{r}_{i_1}$ and $\mathbf{r}_{i_2}$, with each element q_{i_1,i_2} defines as $\mathrm{Cov}(\mathbf{r}_{i_1},\mathbf{r}_{i_2})$.

The problem is then considered as a generalized eigenvalue problem. We apply the Rayleigh-Ritz theorem to address this constrained optimization problem. This approach finds an optimal coefficient vector $\mathbf{w}_1$ to extract the inherent intrinsic component $\mathbf{b}$ that maximizes inter-group correlations while being constrained by intra-group variance.

$$\mathbf{w}_1 = \arg\max_{\mathbf{w}} \frac{\mathbf{w}^T\mathbf{U}\mathbf{w}}{\mathbf{w}^T\mathbf{Q}\mathbf{w}} \tag{6}$$

Typically, this process results in N_I eigenvalues, arranged in descending order. Each eigenvalue is associated with its respective eigenvector. Functionally, these eigenvectors are used as filters to generate MPS-related patterns. To construct the pattern $\mathbf{y}_g$ for the g-th group, the top two eigenvectors $\{\mathbf{w}_1,\mathbf{w}_2\}$ are used as filter coefficients to extract the inherent intrinsic component $\mathbf{b}$. Concurrently, the remaining eigenvectors $\{\mathbf{w}_3,\ldots,\mathbf{w}_{N_I}\}$ are used to approximate the extraction of the external dynamic component $\mathbf{e}$, achieved by subtracting from a randomly selected sequence within the same group, denoted as $rand(\mathbf{x}_g)$. This synthesis is represented as:

$$\mathbf{y}_g = \sum_{s=1}^{N_S}\sum_{i=1}^{N_I}\left(w_{i,s}\mathcal{D}_{g,i,s,:} + w^*_{i,s}\mathcal{D}_{g,i,s,:}\right) - rand(\mathbf{x}_g) \tag{7}$$

Here, the vector $\mathbf{w}_s=[w_{(1,s)},w_{(2,s)},\ldots,w_{(N_I,s)}]\in\{\mathbf{w}_1,\mathbf{w}_2\}$ is the filter coefficients to extract the intrinsic component within the s-th segment, and $\mathbf{w}^*_s=[w^*_{(1,s)},w^*_{(2,s)},\ldots,w^*_{(N_I,s)}]\in\{\mathbf{w}_3,\ldots,\mathbf{w}_{N_I}\}$ is the filter coefficients for the extraction of extrinsic component within the s-th segment. With this synthesis, the resulting pattern represents both inherent and external characteristics.

The proposed method for generating MPS-related patterns $\mathbf{y}_{\mathcal{D}}$ harmonizes intrinsic and dynamic components from various groups. Each group is weighted differently, allowing for individual calculation of both intrinsic and dynamic components. These are then adjusted using group-specific weights. The vector $\mathbf{w}_{g,s}=[w_{(g,1,s)},w_{(g,2,s)},\ldots,w_{(g,N_I,s)}]\in\{\mathbf{w}_1,\mathbf{w}_2\}$ represents the filter coefficients pertinent to the intrinsic component in the s-th segment within the g-th group. Conversely, $\mathbf{w}^*_{g,s}=[w^*_{(g,1,s)},w^*_{(g,2,s)},\ldots,w^*_{(g,N_I,s)}]\in\{\mathbf{w}_3,\ldots,\mathbf{w}_{N_I}\}$ is utilized to approximate the extraction of the external dynamic component. This extraction is achieved by subtracting a randomly selected sequence from any group, denoted as $rand(\mathbf{x}_{\mathcal{D}})$. The method is represented as:

$$\mathbf{y}_{\mathcal{D}}=\sum_{g=1}^{N_G}o_g\left(\sum_{s=1}^{N_S}\sum_{i=1}^{N_I}\left(w_{g,i,s}\mathcal{D}_{g,i,s,:}+w^*_{g,i,s}\mathcal{D}_{g,i,s,:}\right)\right) - rand(\mathbf{x}_{\mathcal{D}}) \tag{8}$$

Algorithm 1: Web Signal Separation and Recombination (*WebSpare*) Algorithm.

Input:
$\mathcal{S}$: The set of payload length sequences of downstream packets;
N_I: The grouping parameter;
N_P: The partitioning parameter;
Output:
$\mathbf{y}_{\text{gen}}$: The generated run-length sequence of MPS packets;
1: Extract the set of MPS-related pattern $\mathcal{X}$ from $\mathcal{S}$ according to (1);
2: Group and partition $\mathcal{X}$ as $\mathcal{D}$ based on N_I and N_P;
3: Calculate inter-group covariance according to (4);
4: Calculate normalization constraint according to (5);
5: Derive eigenvectors as filter coefficients according to (6);
6: Generate $\mathbf{y}_{\text{gen}}$ according to (7), (8);
7: **return** $\mathbf{y}_{\text{gen}}$

where o_g is the weight allocated to the g-th group, under the condition $\sum_{g=1}^{N_G}o_g=1$, ensuring that weights are normalized across groups. By using uniform weights for each group, we generate a balanced pattern encapsulating the collective characteristics of all groups. The workflow for the *WebSpare* algorithm can be summarized as the Algorithm 1.

B. PLeS *Algorithm*

In this subsection, we present the *PLeS* algorithm, which determines the optimal payload insertion length required by packet insertion operation F_I to preserve the spatio-temporal characteristics of the proxy traffic. The *PLeS* algorithm utilizes a queue $\boldsymbol{\theta}$ to gather the payload length of each packet manipulated by packet padding operation F_P within the manipulating downstream flow $Flow_m$. These lengths retain the spatio-temporal characteristics of the target website. If $\boldsymbol{\theta}$ is not empty, the payload length at its front is dequeued and utilized by the packet insertion F_I as the k-th non-MPS payload length. When $\boldsymbol{\theta}$ is depleted, the insertion payload length, designated as the k-th non-MPS payload length vector $\boldsymbol{v}^k$, is computed through a combination of vectors $\boldsymbol{v}^k_1$, $\boldsymbol{v}^k_2$, and $\boldsymbol{v}^k_3$. These represent the k-th non-MPS packet information from $Flow_m$, a randomly selected downstream flow $Flow_r$ in the *WebSpare* algorithm as the reference downstream flow, and their cumulative distance, denoted as $D(k)$, respectively.

The distance $D(k)$ of run-length cumulative payload areas up to the k-th non-MPS packet between $Flow_r$ and $Flow_m$ can be recursively calculated as follows:

$$D(k) = D\left(k-1\right) + S^k_r - S^{k-1}_m = \sum_{i=1}^{k}\left(S^i_r - S^{i-1}_m\right) \tag{9}$$

where S^k_r denotes the run-length cumulative payload area for the k-th non-MPS packet in $Flow_r$, and S^{k-1}_m denotes run-length cumulative payload area for the $(k-1)$-th non-MPS packet in $Flow_m$. And the cumulative payload area S^k can be calculated

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.

as follows:

$$S^k = \varepsilon \cdot \left(c^{k-1} + c^k\right)/2 \tag{10}$$

where ε denotes the maximum payload size. The element c^k belongs to the run-length cumulative payload length sequence $\mathbf{c} = [c^1, c^2, \ldots, c^T]$, which sequentially accumulates the payload lengths of all packets up to the T-th non-MPS packet. Each c^k is calculated recursively to ensure that the cumulative payload at point k includes the lengths of all packets up to and including the k-th non-MPS packet. The calculation of c^k is expressed as follows:

$$c^k = c^{k-1} + \sum_{i=\Phi(k-1)+1}^{\Phi(k)} l_i \tag{11}$$

where $\Phi(k)$ is the position mapping function, and $l_{\Phi(k)}$, the payload length for the k-th non-MPS packet, is extracted.

Then the ideal payload length L_m^k for the k-th non-MPS packet in the manipulated flow can be derived with the cumulative distance $D(k)$. Using the first-order Newton-Cotes closed formula, L_m^k is formulated as follows:

$$D(k) = \varepsilon \cdot \left(2c_m^{k-1} + L_m^k\right)/2 \tag{12}$$

If the above formula yields a negative value for L_m^k, indicating an impractical negative payload length, we set L_m^k to zero for feasibility. Similarly, if L_m^k exceeds MPS, it is capped at MPS to ensure practicability. Consequently, $\boldsymbol{v}_3^k$ denotes the coarse-grained length vector optimally bridging $Flow_m$ and $Flow_r$, expressed as:$\boldsymbol{v}_3^k = (\varepsilon, L_m^k)$.

The estimated payload length vector $\bar{\boldsymbol{v}}^k$ be derived from a combination of $\boldsymbol{v}_1^k$, $\boldsymbol{v}_2^k$, and $\boldsymbol{v}_3^k$. $\boldsymbol{v}_1^k = (\varepsilon, Flow_m^{k-1})$ captures the length of the $(k-1)$-th non-MPS payload in $Flow_m$, considering that the k-th non-MPS packet has not yet arrived. $\boldsymbol{v}_2^k = (\varepsilon, Flow_r^k)$ signifies the known length of the k-th non-MPS payload in $Flow_r$. Utilizing a weighted approach, $\bar{\boldsymbol{v}}^k$ is computed as a linear combination of these vectors:

$$\bar{\boldsymbol{v}}^k = \alpha_1^k \boldsymbol{v}_1^k + \alpha_2^k \boldsymbol{v}_2^k + \alpha_3^k \boldsymbol{v}_3^k \tag{13}$$

where α_1^k, α_2^k, and α_3^k represent corresponding weights that are iteratively adjusted, and k signifies the iteration step.

Once $\bar{\boldsymbol{v}}^k$ is computed, it may not correspond to typical payload lengths seen in legitimate traffic, possibly revealing a distinctive pattern of *QuicCourier*. To address this, the align function adjusts the estimated payload length in the $\bar{\boldsymbol{v}}^k$ to approximate the nearest actual payload length, denoted as $\boldsymbol{v}^k = Align(\bar{\boldsymbol{v}}^k)$, from the non-MPS packets in $Flow_r$, ensuring adherence to genuine traffic features.

After deriving $\boldsymbol{v}^k$, the next step is to fine-tune the weights α_1^{k+1}, α_2^{k+1}, and α_3^{k+1} for the subsequent iteration. This tuning relies on the cosine distance between the vector $\boldsymbol{v}^k$ and the vectors $\boldsymbol{v}_1^k$, $\boldsymbol{v}_2^k$, and $\boldsymbol{v}_3^k$, ensuring that the coefficients are updated in line with the latest payload length adjustments. Normalization of these coefficients scales them to a standardized range, maintaining the proportionality of their influence on the resulting payload length. As a result of this dynamic updating and normalization, the *PLeS* algorithm becomes more adaptable and progressively improves its fitting capabilities. Algorithm 2 shows the *PLeS* procedure.

Algorithm 2: Packet Length Selection (*PLeS*) Algorithm.

Input:
ε: Maximum payload size (MPS);
$\boldsymbol{\theta}$: The collected non-MPS length queue;
$Flow_m$: Manipulated flow;
$Flow_r$: Reference flow;
Output:
l_m^k: The k-th non-MPS payload length in $Flow_m$;

Initialize $\alpha_1^1, \alpha_2^1, \alpha_3^1 = \text{random}(0, 1)$
if $\boldsymbol{\theta}$ *is not empty* **then**
  $l_m^k = \text{dequeue}(\boldsymbol{\theta})$
end
else
  Calculate cumulative distance $D(k)$ between $Flow_r$ and $Flow_m$ according to (9)-(11);
  Derive ideal payload length L_m^k using $D(k)$ according to (12);
  Clamp L_m^k within threshold $[0, \varepsilon]$ as the vector $\boldsymbol{v}_3^k$;
  Calculate estimated vector $\bar{\boldsymbol{v}}^k$ according to (13);
  Align $\bar{\boldsymbol{v}}^k$ to nearest actual payload length as $\boldsymbol{v}^k = (\varepsilon, l_m^k)$;
  Update weights for the next calculation;
end
return l_m^k

V. Experimental Results and Analysis

In this section, we validate the performance of the proposed *QuicCourier* scheme using a real-world dataset. Due to its sustained reliability in real network conditions, our analysis primarily concentrates on its security within both the general and capability-maximizing adversarial models.

A. Experimental Setup

1) Dataset: In response to the absence of publicly available QUIC website traffic datasets, we initiated the collection of target website traffic within a real-world network environment. This was achieved using automated web browsing scripts, which accessed designated websites at 30-second intervals. The data collection process utilized a computer based in Nanjing, China (Windows 10-64 b, 16 GB RAM). Our proxy node, a virtual private server (VPS) located in Los Angeles was configured with 1 CPU and 2 GB of memory. For the proxy protocol, we selected Socks5, known for its widespread usage. Its implementation was based on the golang language. We deployed the proxy server software on the VPS and installed the proxy client software on the local computer. This setup enabled the client to establish a proxy tunnel with the VPS and access web resources via this node. Both the sending and receiving plugins of *QuicCourier* were integrated into the respective server and client proxy programs. Additionally, both legitimate and covert proxy traffic data were collected and stored on the VPS.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.

TABLE III
OVERVIEW OF THE QUIC WEBSITE BROWSING TRAFFIC DATASET

Website	Category	Number of Samples	Duration of Visits
ACM	Education	6,127	57 days
CGsociety	Search	6,161	49 days
Facebook	Social	6,071	52 days
Pinterest	Short-video	6,116	51 days
Xhs	E-commerce	6,157	56 days

Each sample collection process entailed ensuring all web resources were fully loaded, followed by clearing the cache prior to each new collection. It is important to note that the proposed covert channel scheme leverage the spatio-temporal redundancy in characteristics that arise from varying access behaviors to the same website. Consequently, this approach necessitates multiple iterations of sample collection from specifically chosen target QUIC websites. In this study, we selected five websites representative of distinct categories: ACM (education), CGsociety (search), Facebook (social), Pinterest (short video), and Xhs (e-commerce). The comprehensive overview of the dataset is shown in Table III.

For ACM and CGsociety, we utilized Python's natural language toolkit (NLTK) to generate the top 6,000 most commonly used English nouns, simulating diverse keyword searches. For the remaining websites, we simply accessed their homepages. The data collection spanned approximately two months. During this period, we excluded any webpages that failed to load due to poor network conditions. As a result, we amassed approximately 6,000 flows from each of the five targeted websites. Additionally, we also collected 11,000 QUIC samples, which gathered from Alexa Top Ranking websites, to act as background traffic in our general adversarial model.

2) Evaluation Method: The experiments are designed to evaluate the performance of the proposed covert channel under both general and capability-maximizing adversarial models. In the general adversarial model, the warden is unaware of the covert channel scheme and focuses on detecting anomalies within the network traffic. For the covert channel, the goal is to ensure that covert traffic is misclassified as legitimate and remains indistinguishable from its original categories, avoiding detection or reclassification into anomalous categories. In contrast, under the capability-maximizing adversarial model, the warden is fully aware of the covert channel scheme and has access to both legitimate and covert traffic samples. Here, the warden's primary objective is to accurately distinguish between covert and legitimate traffic while minimizing any interference with regular communication.

Since *QuicCourier* is designed to mimic legitimate traffic patterns, network traffic classification tools are well-suited in both adversarial models to evaluate how effectively it blends covert traffic into legitimate flows. We utilized five typical network traffic classification tools: AppS [7], FS-Net [43], DF [44], EBSNN [45], and NeuTic [46]. AppS employs conventional machine learning, while the other four employ deep learning framework.

Furthermore, under the capability-maximizing adversarial model, four covert channel detection tools were employed to determine the presence of covert channels: the Kolmogorov-Smirnov (KS) test [47], ϵ-similarity test [48], NetBeacon [49], and FlowLens [50]. The KS-test measures the maximum difference between empirical distribution functions to compare the packet length distributions. The ϵ-similarity test evaluates the temporal relationships within packet sequences by calculating the relative differences between consecutive packet lengths. NetBeacon extracts features at both the packet level (e.g., packet length, packet timing) and the flow level (e.g., mean and variance of packet lengths and inter-packet delays) and uses a Random Forest classifier for detection. FlowLens focuses on the distributions of packet lengths and inter-packet arrival times, applying quantization and feature selection techniques, and performs detection using an XGBoost classifier.

To validate the effectiveness of *QuicCourier*, we compare it with two variant schemes, demonstrating that its superior undetectability arises from the efficient utilization of traffic redundancy through the proposed characterization and generative modeling approach. *QuicVanilla* is a baseline method that does not leverage the dynamic features of traffic and instead applies fixed-length padding to each downstream packet. *QuicMarkov*, on the other hand, incorporates a first-order Markov transition model to capture the redundancy in website browsing traffic. It adjusts run-length values and non-MPS payload lengths based on preceding packets, utilizing a set of dynamically changing transition matrices, the utilized transition matrice strategy of which have been applied in traffic analysis and traffic obfuscation [51], [52].

These experiments were carried out on an Intel i9-12900 K CPU, 64 GB RAM, and an NVIDIA GeForce RTX 3090 with 24 GB GPU memory. We adopted a 5-fold cross-validation method for each experiment.

3) Evaluation Metric: To accommodate the requirement of a predefined number of packets in most network traffic classification tools, we standardized the number of packets per flow to be $N = 3000$ in the experiments. The proposed network covert channel scheme also uses N as the available carrier packet number per flow to ensure the fairness. Unlike conventional network covert channels, which have a defined covert transmission rate, the proposed scheme embeds a fixed amount of M bytes per flow. Consequently, with each flow comprising N carrier packets, the average bytes per packet (bpp), M/N, serves as the metric for measuring the capacity of the network covert channel.

To evaluate the similarity between legitimate and covert traffic samples, the perplexity (PPL) is first utilized in our experiment. The perplexity is a commonly used metric for evaluating language model performance [53]. It is calculated based on the average logarithmic probability assigned to each word in a text. Considering that the request-response pattern in website browsing mirrors communication behavior, each packet in this scenario captures specific interaction details. Therefore, the length of each packet's payload can be likened to a word. The perplexity can be calculated by:

$$PPL = 2^{-\frac{1}{n}\sum_{j=2}^{n}\log_2 p(z_j|\mathbf{z}_{1:j-1})} \tag{14}$$

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.

where $\mathbf{z} = [z_1, z_2, \ldots, z_n]$ denotes the payload length sequence of a network flow, and $\mathbf{z}_{1:j-1}$ indicates the sequence of payload length up to the $(j-1)$-th packet. Generally, a lower perplexity value indicates a higher degree of similarity between the flows.

Furthermore, we also employed the widely used Jensen-Shannon divergence (JSD) as a similarity metric to assess the similarity between legitimate and covert channel samples. The calculation of JSD is as follows:

$$JSD = \frac{1}{2} D_{KL}\left(P_\alpha \| \frac{P_\alpha + P_\beta}{2}\right) + \frac{1}{2} D_{KL}\left(P_\beta \| \frac{P_\alpha + P_\beta}{2}\right) \quad (15)$$

where P_α and P_β are the statistical distributions of network flow α and β, respectively. $D_{KL}(\cdot\|\cdot)$ denotes the KL divergence.

Beyond similarity metrics, the actual results of network traffic identification tools are crucial in assessing security. We utilize three identification metrics for both the general and capability-maximizing adversarial models. In the general model, the Original Detection Rate (ODR) quantifies the classifier's detection rate for original web browsing traffic. Conversely, the Secret Evasion Rate (SER) gauges the classifier's evasion rate for covert traffic, evaluating the degree to which such traffic is categorized within its original class. The formulations for ODR and SER are delineated as follows:

$$ODR = \frac{2 \times P_l \times R_l}{P_l + R_l}$$
$$SER = \frac{2 \times P_c \times R_c}{P_c + R_c} \quad (16)$$

where P_l and R_l denote the precision and recall for the legitimate traffic, while P_c and R_c correspond to the precision and recall for the covert traffic classified as its legitimate class. A higher SER relative to ODR signifies a more effective deception of the warden.

In the capability-maximizing model, we utilize the accuracy (Acc) metric to evaluate the resistance of covert channels against 2-class traffic identification tools. A lower accuracy value is more desirable for resisting covert channel detection.

$$Acc = \frac{TP + TN}{TP + TN + FP + FN} \quad (17)$$

where the true positive (TP) and false negative (FN) are used to indicate correctly identified and misclassified covert traffic, respectively, while the true negative (TN) and false positive (FP) pertained to the legitimate traffic. In addition, we use the Receiver Operating Characteristics (ROC) curve and the Area Under the ROC Curve (AUC) to evaluate the resistance of covert channels against machine learning-based covert traffic detection tools, where the ROC provides a visual representation and the AUC measures the overall discriminative ability to distinguish covert traffic from legitimate traffic

B. The Selection of Global Parameters

In our covert channel scheme, both the number of sequences per group and the size of each segment within sequences are predetermined as global parameters (N_I, N_P). We conducted a thorough analysis of these two parameters, evaluating various values to determine the most optimal configurations. This evaluation was supported by similarity metrics.

TABLE IV
THE SIMILARITY DISTANCE WITH DIFFERENT N_I VALUES

Website	*Metric*	N_I				
		15	20	25	30	35
ACM	ΔPPL	0.34	0.30	**0.23**	0.28	0.24
	ΔJSD	0.00008	0.00003	**0.00002**	0.00003	0.00004
CGsociety	ΔPPL	**2.55**	2.59	2.56	2.61	2.59
	ΔJSD	**0.00020**	0.00023	0.00021	0.00022	0.00023
Facebook	ΔPPL	0.64	0.63	**0.62**	0.63	0.63
	ΔJSD	0.00080	**0.00079**	0.00080	0.00083	0.00082
Pinterest	ΔPPL	2.55	2.51	**2.42**	2.68	2.50
	ΔJSD	**0.00023**	0.00027	0.00026	0.00035	0.00042
Xhs	ΔPPL	0.39	0.41	**0.38**	0.39	0.40
	ΔJSD	0.00016	0.00017	0.00016	0.00016	**0.00015**

The approach for parameter selection in the experiments included distinct tests to meticulously adjust the values of N_I and N_P. For each website, we employed 3,000 samples to generate MPS-related patterns. In addition, for comparative analysis, we replayed a different set of 3,000 samples: 1,000 of these samples were designated as covert traffic, another 1,000 as comparison samples, and the final 1,000 as benchmark samples. Importantly, to highlight the variations in traffic patterns, we manipulated the entire data stream without imposing any capacity constraints.

The perplexity differentiation between covert traffic PPL_c and legitimate traffic PPL_l is evaluated by calculating the difference in their respective mean perplexity values, which is expressed as follows:

$$\Delta PPL = mean(PPL_c) - mean(PPL_l) \quad (18)$$

Although the perplexity provides insights into sequence distance, it does not comprehensively capture the statistical variations in payload lengths between covert and legitimate samples. To remedy this, we incorporate the calculation of the JSD for both sample types, expressed as JSD_c for covert traffic and JSD_l for legitimate traffic:

$$\Delta JSD = mean(JSD_c) - mean(JSD_l) \quad (19)$$

To ascertain the optimal value for N_I, we tested values of 15, 20, 25, 30, and 35, computing both ΔPPL and ΔJSD for each. As indicated in Table IV, the results were derived with N_P set at 20. The analysis showed a relatively stable pattern across various N_I values, as demonstrated by the minimal variations in ΔPPL and ΔJSD. Notably, the fluctuation in ΔJSD was marginal, exhibiting differences on the order of 10^{-5}, while ΔPPL remained within a 10^{-2} range. This uniformity suggests that the choice of N_I has a limited effect on the inconspicuousness of the traffic. However, a higher N_I allows for a wider range of filter coefficients, thereby enabling the creation of more varied patterns. Among the evaluated values, $N_I = 15$ and $N_I = 25$ showed the most favorable results, with the lowest metric counts being 3 and 5, respectively. We chose $N_I = 25$ for a balance between dynamic pattern generation and traffic inconspicuousness.

Furthermore, to investigate the impact of varying N_P values on ΔPPL and ΔJSD, we conducted a series of experiments

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.

TABLE V
THE SIMILARITY DISTANCE WITH DIFFERENT N_P VALUES

Website	*Metric*	N_P				
		5	10	15	20	25
ACM	ΔPPL	0.56	0.38	0.46	**0.23**	0.33
	ΔJSD	0.00020	0.00006	0.00007	**0.00002**	0.00005
CGsociety	ΔPPL	2.89	2.85	**2.43**	2.56	2.72
	ΔJSD	0.00026	0.00020	**0.00017**	0.00021	0.00022
Facebook	ΔPPL	0.62	0.62	**0.57**	0.62	0.59
	ΔJSD	0.00062	0.00070	0.00061	0.00080	**0.00060**
Pinterest	ΔPPL	3.25	3.39	**2.39**	2.42	2.65
	ΔJSD	0.00042	0.00069	**0.00018**	0.00026	0.00020
Xhs	ΔPPL	0.55	0.57	**0.38**	**0.38**	0.53
	ΔJSD	0.00024	0.00023	**0.00010**	0.00016	0.00021

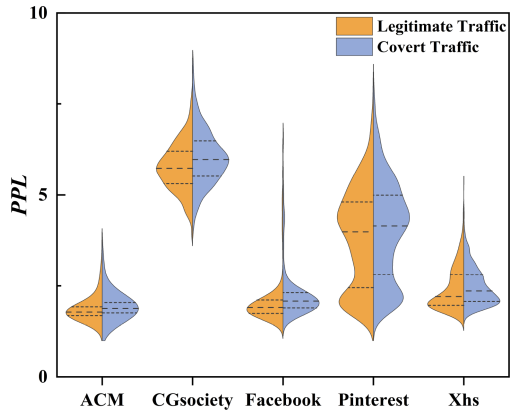


Fig. 5. Comparison of similarity between covert and legitimate traffic using perplexity.

with N_P ranging from 5 to 25, increasing in increments of 5. The outcomes of these experiments are thoroughly outlined in Table V. A notable observation from the results is that ΔPPL exhibited significant variation with different N_P values, while ΔJSD remained comparatively stable. A key finding is that for most websites, an N_P range of 15 to 20 was associated with the highest degree of imperceptibility. This indicates that segmenting sequences with an N_P value in this range is conducive to creating more effective patterns. Based on these insights, we selected $N_P = 15$ as the standard parameter for the *WebSpare* algorithm.

In our analysis of parameter selection, we discovered that PPL is a more effective metric than JSD for assessing the inconspicuousness of covert traffic samples. To extend this analysis, we employed PPL to contrast covert and legitimate traffic samples, particularly under varying *bpp* conditions. Fig. 5 displays this comparison through a violin plot, clearly delineating the PPL differences between covert and legitimate traffic samples across five selected websites. The results reveal that each website exhibits slightly higher PPL for traffic containing secret data, indicating a minor departure from legitimate traffic. Notably, CGsociety shows the most significant PPL disparity, as reflected in the quartile comparisons. Nonetheless, it is important to acknowledge that all covert traffic embedded with secret data remains closely analogous to their legitimate samples in terms of PPL. The observed divergence is within an acceptable margin, demonstrating that these traffic samples effectively retain the spatio-temporal characteristics of the corresponding legitimate traffic. Consequently, the resemblance poses a significant challenge for a warden attempting to distinguish between covert and legitimate traffic based solely on perplexity.

C. Undetectability in the General Adversarial Model

In the general adversarial model, the warden functions without awareness of the proposed covert channel. The warden's main goal is to identify any anomaly within the network traffic. This involves the continuous monitoring of all network activities using sophisticated traffic identification tools. These tools excel in recognizing different websites, even when proxies are employed to alter the traffic's statistical properties. For the proposed *QuicCourier* scheme, the objective is to ensure that covert traffic samples are misclassified as part of their original categories and not marked as unusual or attributed to different categories.

To validate the effectiveness in the general adversarial model, we generated MPS-related patterns using 3,000 traffic samples for each website. In a separate set of 3,000 samples, we designated 400 for secret data transmission, used another 400 as legitimate samples, and allocated the remaining 2,200 as part of the benchmark dataset for training classifiers. To mimic a realistic network environment, we augmented this dataset with 11,000 additional QUIC-web traffic samples, representing a wide spectrum of background traffic. This method resulted in a comprehensive training dataset totaling 22,000 samples, ensuring a balanced distribution between targeted website traffic and other traffic types. After classifier training, we employed 400 legitimate and 400 covert traffic samples to assess the performance of the classifiers.

Each traffic classification tool exhibits different ODR and SER across various websites and covert channel schemes. For a clear comparison of overall performance, we calculate the average ODR across all websites for each classifier and the average SER for each covert channel scheme. The symbol "↓" indicates the percentage decrease in the covert channel scheme's performance SER relative to the average ODR of legitimate traffic. As shown in Table VI, all five classifiers exhibit high ODR for legitimate website traffic, with values exceeding 98%, indicating their strong ability to accurately identify client activities. Across various traffic classification tools, *QuicCourier*'s covert traffic remains nearly indistinguishable from legitimate traffic, with its SER closely matching the ODR. This demonstrates its ability to maintain the spatio-temporal characteristics of web traffic, effectively evading detection.

Classifiers like AppS, which utilizes a simpler random forest model, and DF, which relies on packet direction sequences, may not accurately capture subtle differences. On the other hand, *QuicVanilla* is highly vulnerable to detection by advanced classifiers such as NeuTic. Its vulnerability stems from its simplistic approach, which introduces detectable anomalies and renders the covert traffic distinguishable from legitimate patterns. While *QuicMarkov* achieves better undetectability than *QuicVanilla*, its reliance on a first-order Markov model limits its ability to fully capture the dynamic spatio-temporal characteristics of traffic. This limitation makes *QuicMarkov* more detectable than

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
THE ODRS FOR LEGITIMATE TRAFFIC AND SERS FOR COVERT TRAFFIC USING ADVANCED TRAFFIC CLASSIFICATION TOOLS UNDER THE GENERAL ADVERSARIAL MODEL

	Website	ODR	SER											
			QuicVanilla				*QuicMarkov*				*QuicCourier*			
			2 *bpp*	3 *bpp*	5 *bpp*	7 *bpp*	2 *bpp*	3 *bpp*	5 *bpp*	7 *bpp*	2 *bpp*	3 *bpp*	5 *bpp*	7 *bpp*
AppS [7]	ACM	0.996	0.995	0.994	0.994	0.993	0.990	0.993	0.992	0.999	0.994	0.994	0.995	0.997
	CGsociety	1.000	0.997	0.998	0.923	0.732	0.991	0.999	0.988	0.995	1.000	0.999	1.000	1.000
	Facebook	0.998	0.996	0.993	0.995	0.992	0.998	0.994	0.991	0.994	1.000	0.999	0.999	1.000
	Pinterest	0.998	0.996	0.994	0.993	0.999	0.990	0.993	0.991	0.996	0.995	0.992	0.995	0.998
	Xhs	0.998	0.992	0.990	0.995	0.989	0.998	0.994	0.994	0.993	0.996	0.995	0.997	0.996
	Average	**0.998**	**0.978 (↓ 0.020)**				**0.994 (↓ 0.004)**				**0.997 (↓ 0.001)**			
DF [44]	ACM	0.972	0.976	0.976	0.976	0.976	0.974	0.966	0.963	0.959	0.976	0.965	0.966	0.968
	CGsociety	0.995	0.996	0.996	0.996	0.996	0.997	0.989	0.986	0.993	0.997	0.997	0.988	0.997
	Facebook	0.980	0.977	0.977	0.977	0.977	0.973	0.978	0.986	0.975	0.976	0.974	0.981	0.984
	Pinterest	0.967	0.973	0.973	0.973	0.973	0.966	0.965	0.965	0.975	0.972	0.963	0.955	0.972
	Xhs	0.989	0.982	0.982	0.982	0.982	0.998	0.981	0.981	0.986	0.987	0.986	0.990	0.982
	Average	**0.981**	**0.981 (↓ 0.000)**				**0.978 (↓ 0.003)**				**0.979 (↓ 0.002)**			
FS-Net [43]	ACM	0.994	0.738	0.688	0.685	0.668	0.985	0.984	0.993	0.984	0.995	0.985	0.987	0.982
	CGsociety	0.994	0.990	0.858	0.735	0.399	0.995	0.994	0.990	0.995	0.992	0.989	0.997	0.993
	Facebook	0.995	0.991	0.597	0.031	0.013	0.981	0.994	0.995	0.989	0.996	0.991	0.991	0.988
	Pinterest	0.993	0.992	0.972	0.918	0.278	0.983	0.988	0.981	0.990	0.991	0.990	0.989	0.990
	Xhs	0.999	0.965	0.818	0.674	0.669	0.983	0.979	0.998	0.989	0.998	0.997	0.998	0.999
	Average	**0.995**	**0.684 (↓ 0.311)**				**0.989 (↓ 0.006)**				**0.992 (↓ 0.003)**			
EBSNN [45]	ACM	0.994	0.988	0.948	0.983	0.960	0.995	0.994	0.995	0.998	0.995	0.988	0.992	0.999
	CGsociety	0.998	0.748	0.354	0.117	0.012	0.995	0.995	0.996	0.995	0.999	0.998	0.998	0.998
	Facebook	0.995	0.985	0.976	0.728	0.139	0.995	0.998	0.991	0.990	0.997	0.996	0.998	0.998
	Pinterest	0.992	0.903	0.763	0.610	0.388	0.993	0.999	0.996	0.994	0.995	0.993	0.988	0.995
	Xhs	0.998	0.893	0.893	0.818	0.815	0.995	0.990	0.999	0.989	0.999	0.989	0.992	0.997
	Average	**0.995**	**0.701 (↓ 0.294)**				**0.995 (↓ 0.000)**				**0.995 (↓ 0.000)**			
NeuTic [46]	ACM	0.995	0.408	0.263	0.047	0.012	0.995	0.991	0.981	0.991	0.997	0.996	0.997	0.998
	CGsociety	0.999	0.968	0.780	0.708	0.510	0.985	0.984	0.986	0.984	1.000	1.000	0.998	1.000
	Facebook	0.995	1.000	0.315	0.173	0.005	0.994	0.984	0.988	0.980	0.999	0.995	0.996	0.998
	Pinterest	0.992	0.973	0.968	0.956	0.948	0.991	0.992	0.994	0.993	0.995	0.993	0.989	0.992
	Xhs	0.998	0.665	0.453	0.286	0.263	0.994	0.989	0.984	0.998	1.000	1.000	0.998	0.999
	Average	**0.996**	**0.535 (↓ 0.461)**				**0.989 (↓ 0.007)**				**0.996 (↓ 0.000)**			

QuicCourier, particularly as the *bpp* value increases. The comparative results demonstrate that, without explicit knowledge of the covert channel, advanced packet manipulation strategies significantly reduce the likelihood of covert traffic detection, enabling *QuicCourier* to evade classification.

D. Undetectability in the Capability-Maximizing Adversarial Model

In the capability-maximizing adversarial model, where the warden possesses detailed knowledge of *QuicCourier*, the primary objective is to accurately differentiate between covert and legitimate traffic without impeding regular communication. Utilizing a balanced dataset comprising both legitimate and covert samples for each target website, the warden develops specialized classifiers aimed at identifying covert traffic. To optimize warden's effectiveness, we postulate the existence of distinct classifiers for each target website, equipped with the ability to discern covert traffic based on the traffic samples.

In these experiments, 3,000 flows per website were utilized for generating MPS-related patterns. For testing purposes, an additional 3,000 flows were employed, with half designated for transmitting secret data (positive samples) and the other half serving as control (negative samples). In the two-class classification scenario, we used an 8:2 training-to-testing split, with accuracy as the primary performance metric.

For legitimate traffic, each classifier demonstrates near-random guess accuracy (around 50%) since it attempts to differentiate between two sets of legitimate traffic samples. To evaluate overall undetectability, we calculate the average accuracy across websites for each covert channel scheme and classifier. The "↑" symbol indicates the increase in detection accuracy for covert traffic compared to legitimate traffic. As shown in Table VII, *QuicVanilla* is almost entirely exposed, with four advanced classifiers detecting its covert traffic with over 95% accuracy. However, because *QuicVanilla*'s inherent padding does not disrupt packet direction sequences, it remains indistinguishable under the DF classifier. In contrast, *QuicMarkov* performs significantly better, showing an average 20% reduction in detection accuracy compared to *QuicVanilla*. However, as the *bpp* value increases, the detection rates for *QuicMarkov* also rise, as its first-order Markov model fails to capture longer-dependency spatio-temporal characteristics required to conceal larger amounts of secret data effectively. These findings highlight the importance of evaluating covert channel schemes across multiple classifiers to comprehensively assess their robustness.

On the other hand, *QuicCourier* demonstrates substantial declines in detection accuracy, approaching random chance predictions. For instance, AppS achieves a maximum accuracy of only 55.2%, indicating its inability to detect *QuicCourier*. Deep learning-based classifiers such as DF, FS-Net, and EBSNN

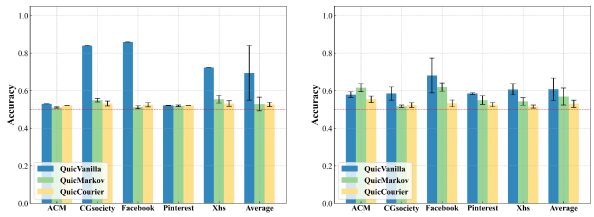
Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
THE CLASSIFICATION ACCURACY FOR COVERT TRAFFIC USING TRAFFIC CLASSIFICATION TOOLS UNDER THE CAPABILITY-MAXIMIZING ADVERSARIAL MODEL

	Website	Accuracy											
		Legitimate	*QuicVanilla*				*QuicMarkov*				*QuicCourier*			
			2 *bpp*	3 *bpp*	5 *bpp*	7 *bpp*	2 *bpp*	3 *bpp*	5 *bpp*	7 *bpp*	2 *bpp*	3 *bpp*	5 *bpp*	7 *bpp*
AppS [7]	ACM	0.504	0.997	0.996	0.999	1.000	0.602	0.598	0.607	0.635	0.522	0.515	0.521	0.512
	CGsociety	0.499	0.861	0.862	0.862	0.862	0.493	0.530	0.534	0.536	0.505	0.501	0.497	0.507
	Facebook	0.501	0.999	0.999	0.999	0.999	0.607	0.606	0.620	0.629	0.505	0.537	0.536	0.545
	Pinterest	0.492	0.979	0.979	0.978	0.979	0.584	0.610	0.615	0.625	0.509	0.519	0.524	0.552
	Xhs	0.499	0.940	0.940	0.940	0.940	0.568	0.605	0.635	0.656	0.501	0.482	0.482	0.507
	Average	**0.499**		**0.956 (↑ 0.457)**				**0.595 (↑ 0.096)**				**0.514 (↑ 0.015)**		
DF [44]	ACM	0.518	0.501	0.501	0.501	0.501	0.616	0.5988	0.681	0.642	0.655	0.634	0.653	0.618
	CGsociety	0.498	0.500	0.500	0.500	0.500	0.613	0.659	0.639	0.694	0.549	0.638	0.640	0.601
	Facebook	0.505	0.504	0.504	0.504	0.504	0.630	0.638	0.671	0.719	0.500	0.510	0.511	0.504
	Pinterest	0.505	0.499	0.499	0.499	0.499	0.618	0.680	0.694	0.758	0.503	0.498	0.514	0.504
	Xhs	0.502	0.503	0.503	0.503	0.503	0.631	0.642	0.670	0.794	0.555	0.550	0.553	0.544
	Average	**0.506**		**0.501 (↓ 0.004)**				**0.664 (↑ 0.158)**				**0.562 (↑ 0.056)**		
FS-Net [43]	ACM	0.528	0.987	0.994	0.997	1.000	0.696	0.706	0.736	0.738	0.652	0.636	0.524	0.592
	CGsociety	0.545	0.976	0.977	0.986	0.997	0.705	0.767	0.789	0.808	0.587	0.547	0.559	0.571
	Facebook	0.543	0.983	0.983	0.994	0.999	0.558	0.662	0.793	0.814	0.507	0.522	0.521	0.509
	Pinterest	0.529	0.991	0.980	0.996	0.991	0.628	0.700	0.836	0.796	0.511	0.509	0.515	0.508
	Xhs	0.530	0.993	0.996	0.996	0.996	0.634	0.629	0.693	0.805	0.603	0.550	0.622	0.544
	Average	**0.535**		**0.991 (↑ 0.456)**				**0.725 (↑ 0.190)**				**0.554 (↑ 0.019)**		
EBSNN [45]	ACM	0.540	0.979	0.980	0.982	1.000	0.679	0.676	0.793	0.863	0.548	0.549	0.585	0.598
	CGsociety	0.508	0.985	0.987	0.982	0.991	0.575	0.597	0.674	0.695	0.575	0.597	0.500	0.504
	Facebook	0.502	0.994	0.999	0.984	0.990	0.597	0.765	0.782	0.820	0.561	0.590	0.598	0.633
	Pinterest	0.510	0.987	0.984	0.988	0.989	0.612	0.677	0.683	0.821	0.511	0.502	0.511	0.551
	Xhs	0.512	0.978	0.982	0.991	0.998	0.504	0.554	0.619	0.780	0.546	0.593	0.591	0.598
	Average	**0.514**		**0.988 (↑ 0.474)**				**0.688 (↑ 0.174)**				**0.562 (↑ 0.048)**		
NeuTic [46]	ACM	0.468	0.995	0.993	0.995	1.000	0.833	0.812	0.822	0.825	0.651	0.635	0.660	0.717
	CGsociety	0.498	0.989	0.991	0.994	0.995	0.782	0.758	0.808	0.815	0.572	0.560	0.595	0.565
	Facebook	0.499	0.991	0.989	0.994	0.996	0.742	0.780	0.781	0.812	0.548	0.632	0.723	0.737
	Pinterest	0.505	0.987	0.992	0.992	0.993	0.830	0.830	0.803	0.803	0.558	0.586	0.533	0.648
	Xhs	0.494	0.992	0.995	0.996	1.000	0.780	0.793	0.793	0.780	0.640	0.600	0.662	0.653
	Average	**0.493**		**0.993 (↑ 0.500)**				**0.799 (↑ 0.306)**				**0.624 (↑ 0.131)**		

perform slightly better, with average detection accuracies near 55%. Specifically, DF achieves the highest accuracy with websites like ACM and CGsociety, FS-Net performs better with ACM and Xhs, and EBSNN performs best with Facebook. Notably, NeuTic delivers the strongest detection performance, particularly for ACM and Facebook traffic, achieving an average accuracy of approximately 60% when the average secret data transmitted per browsing session is around 12 KB. However, it is important to note that NeuTic incurs significantly higher computational costs compared to other classifiers.

Although NeuTic's performance exceeds the average, it falls below 65% accuracy for most websites, except in the cases of ACM and Facebook. This exception can be linked to the lower level of dynamism in these websites, leading to more uniform and predictable traffic patterns. Specifically, ACM, with its academic orientation, consistently loads static resources like thumbnails, CSS, and font files. This consistency persists regardless of the search keywords used, resulting in more predictable traffic patterns. Conversely, Facebook, despite its capacity for dynamic resources through various multimedia contents, exhibits a degree of uniformity in the dataset. This uniformity is primarily due to the data being sourced from a single account. The limited diversity in friends' network and the static nature of recommended content contribute to less variable traffic patterns on Facebook.



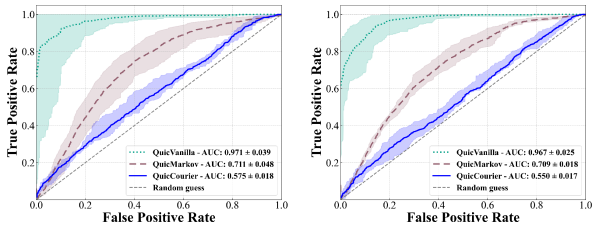
(a) (b)

Fig. 6. Detection accuracy under capability-maximizing adversarial model for statistical-based detection tools: (a) KS-test; (b) ϵ-similarity test.

In addition to encrypted traffic classification tools, we also employ statistical-based and machine learning-based covert channel detection methods to evaluate the proposed scheme. The statistical-based detection methods include the KS-test and the ϵ-similarity test, while the machine learning-based detection methods consist of NetBeacon and FlowLens.

Fig. 6 illustrates the average detection accuracy for each website under varying *bpp* values. *QuicCourier* demonstrates superior resistance to both detection approaches compared to its variants. *QuicVanilla* exhibits the highest detection accuracy, with an average of 69.5% in the KS-test and 60.7% in the ϵ-similarity test. Although *QuicMarkov* performs comparably to *QuicCourier* in the KS-test, it is more easily detected by the

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 7. Mean ROC curves for machine learning-based detection tools: (a) FlowLens; (b) NetBeacon.

ϵ-similarity test. This result suggests that *QuicMarkov* struggles to consistently maintain dynamic patterns, reducing its effectiveness in evading ϵ-similarity-based detection.

Despite achieving relatively high detection rates for *QuicVanilla* on certain websites, both statistical-based detection methods display inconsistent performance across different scenarios. To further evaluate *QuicCourier*'s undetectability, we employ two machine learning-based covert channel detection tools, NetBeacon and FlowLens. For a comprehensive evaluation, as shown in Fig. 7, we apply various thresholds for these tree-based classifiers to generate ROC curves and calculate AUC values.

Fig. 7(a) and (b) depict the mean ROC curves for the three covert channel schemes, showing that both detection tools perform similarly across all schemes. *QuicVanilla* has the highest detection rate, with an AUC of 0.97, followed by *QuicMarkov*, which achieves an AUC of approximately 0.71. *QuicCourier* is the most difficult scheme to detect, with an AUC of around 0.56, indicating strong undetectability. Furthermore, *QuicCourier*'s detection results remain consistent across all websites. The similarity in ROC curves and AUC values demonstrates *QuicCourier*'s ability to maintain dynamic traffic patterns across different websites, further enhancing its effectiveness in evading detection.

E. Discussion

Experimental results under different adversarial models demonstrate that detecting *QuicCourier* remains a significant challenge for wardens, even in the capability-maximizing adversarial model. In this scenario, wardens are equipped with labeled covert traffic samples and advanced detection tools, yet their ability to accurately identify *QuicCourier* traffic remains limited. Despite these promising results, *QuicCourier* faces two primary limitations when deployed in real-world Internet environments.

The first limitation lies in adapting to the heterogeneous and dynamic statistical redundancy of traffic across different websites. In open-world scenarios, websites exhibit substantial heterogeneity, resulting in significant variations in the statistical characteristics of their traffic. Additionally, traffic patterns are highly dynamic and vary widely across websites, requiring individualized modeling of traffic patterns for each target website to construct effective covert channels. Experimental results indicate that websites with highly dynamic content offer significantly greater secure embedding capacities compared to those with static content. However, accurately assessing the secure capacity of each website remains inherently challenging.

In practical applications, this limitation necessitates prioritizing websites with dynamic traffic patterns to ensure secure covert channels while reducing covert transmission rates for static websites. Furthermore, even for the same website, variations in traffic characteristics caused by content updates or access configurations complicate modeling. Addressing this requires collecting more initial traffic samples and continuously updating datasets to maintain accurate and comprehensive traffic models. Thus, adapting to the heterogeneity and dynamic nature of website traffic redundancy remains a core challenge for *QuicCourier*.

The second limitation is the reliance on web browsing behavior, which inherently requires user involvement. The establishment of the proposed covert channel depends on web browsing activities, necessitating some level of user participation in real-world applications. Secret data is transmitted by embedding message fragments into individual network flows, requiring users to actively browse websites.

Mitigating this limitation involves selecting websites with high-frequency automatic refresh mechanisms, which are already common among many popular websites. Such mechanisms significantly reduce the need for manual clicks or refreshes during covert communication. Additionally, browser-based automatic refresh configurations can be utilized to further minimize user involvement. These strategies help alleviate the dependence on user behavior, making the proposed scheme more practical for real-world deployment.

Despite these limitations, *QuicCourier*'s security is bolstered by the unpredictable and complex nature of the Internet. First, the vastness of the Internet makes exhaustive monitoring by wardens impractical [54]. Many websites accessed by *QuicCourier* are likely to evade detection due to the sheer scale of the network. Experimental results under the general adversarial model confirm the inherent challenges wardens face in detecting *QuicCourier* under typical scenarios. Second, the problem of data imbalance presents an additional hurdle [55]. In real-world scenarios, it is unlikely that wardens can gather a comprehensive dataset with sufficient samples of *QuicCourier* traffic. As *QuicCourier*'s usage increases, its traffic will blend more seamlessly with legitimate traffic, contaminating detection datasets and further complicating detection efforts. This scenario adds significant complexity to the warden's task in an uncontrolled, real-world environment, in sharp contrast to laboratory conditions. By leveraging the Internet's scale and unpredictability, *QuicCourier* gains additional layers of security, making it a robust and practical solution for covert communication.

Beyond the widespread adoption of the QUIC protocol, which offers a vast range of potential carriers, *QuicCourier*'s effectiveness lies in its innovative utilization of spatio-temporal statistical redundancy inherent in QUIC web browsing traffic. By employing a characterization generative model for web resource patterns, it ensures consistency with legitimate traffic, even against advanced statistical detection techniques. Furthermore, the extensive state information within QUIC's encrypted structure minimizes the risk of detectable protocol anomalies. In

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.

contrast, covert channels built on other protocols often fail to preserve behavioral consistency. For instance, WEASEL embeds secret data into DNS elements such as domain names, but these domain names are non-existent in DNS records and deviate from legitimate linguistic patterns, exhibiting unnatural character distributions that make them vulnerable to statistical detection [56]. Similarly, IPv6teal embeds data in the Flow Label field of the IPv6 header, but frequent and inconsistent modifications across packets disrupt normal IPv6 traffic patterns, introducing detectable anomalies [57]. Moreover, once these covert channels are identified, simple countermeasures such as traffic normalization can effectively neutralize them without affecting legitimate communications. By contrast, *QuicCourier*'s strategic use of redundancy, combined with its ability to maintain comprehensive behavioral consistency, significantly enhances its stealth and resilience, distinguishing it from traditional covert channels.

VI. Conclusion

In this study, we introduce *QuicCourier*, a novel covert channel that utilizes the dynamic spatio-temporal characteristics of QUIC-web browsing traffic for secret data transmission. This method manipulates QUIC packets within proxy services while maintaining standard proxy communications. The inherent encapsulation capabilities of proxy services further support such manipulations. *QuicCourier* exhibits robustness against various traffic analysis tools under both general and capability-maximizing adversarial models. This covert channel framework offers a way to seamlessly integrate various web services with web proxies, enabling high-capacity covert communications in real networks.

This work represents an initial exploration into leveraging proxy services and traffic redundancy for covert communication, currently focusing on web browsing behavior. While our implementation demonstrates the effectiveness of utilizing statistical redundancy in web traffic, similar redundancy patterns also exist in other network activities, such as mobile applications and streaming multimedia. These areas suggest promising directions for future research, expanding the applicability of the proposed method beyond web browsing.

References

[1] B. W. Lampson, "A note on the confinement problem," *Commun. ACM*, vol. 16, no. 10, pp. 613–615, 1973.

[2] Y. Shen, L. Huang, X. Lu, and W. Yang, "A novel comprehensive steganalysis of transmission control protocol/internet protocol covert channels based on protocol behaviors and support vector machine," *Secur. Commun. Netw.*, vol. 8, no. 7, pp. 1279–1290, 2015.

[3] J. K. H. Iv, M. Georgiou, A. J. Malozemoff, and T. Shrimpton, "Security foundations for application-based covert communication channels," in *Proc. IEEE Symp. Secur. Priv.*, 2022, pp. 1971–1986.

[4] L. Caviglione, M. Podolski, W. Mazurczyk, and M. Ianigro, "Covert channels in personal cloud storage services: The case of dropbox," *IEEE Trans. Ind. Inform.*, vol. 13, no. 4, pp. 1921–1931, Aug. 2017.

[5] D. Barradas, N. Santos, L. Rodrigues, and V. Nunes, "Poking a hole in the wall: Efficient censorship-resistant internet communications by parasitizing on webrtc," in *Proc. Conf. Comput. Commun. Secur.*, 2020, pp. 35–48.

[6] Z. Sun and V. Shmatikov, "Telepath: A minecraft-based covert communication system," in *Proc. IEEE Symp. Secur. Priv.*, 2023, pp. 2223–2237.

[7] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Robust smartphone app identification via encrypted network traffic analysis," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 1, pp. 63–78, Jan. 2018.

[8] B. Gao, W. Liu, G. Liu, and F. Nie, "Resource knowledge-driven heterogeneous graph learning for website fingerprinting," *IEEE Trans. Cogn. Commun. Netw.*, vol. 10, no. 3, pp. 968–981, Jun. 2024.

[9] F. Iglesias, F. Meghdouri, R. Annessi, and T. Zseby, "CCgen: Injecting covert channels into network traffic," *Secur. Commun. Netw.*, vol. 2022, no. 1, 2022, Art. no. 2254959.

[10] M. E. A. Zhan, "Website-aware protocol confusion network for emergent HTTP/3 website fingerprinting," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 2427–2439, 2023.

[11] M. Wang, A. Kulshrestha, L. Wang, and P. Mittal, "Leveraging strategic connection migration-powered traffic splitting for privacy," in *Proc. Privacy Enhancing Technol.*, 2022, pp. 498–515.

[12] J.-P. Smith, P. Mittal, and A. Perrig, "Website fingerprinting in the age of quic," in *Proc. Privacy Enhancing Technol.*, 2021, pp. 48–69.

[13] T. Schmidbauer and S. Wendzel, "SoK: A survey of indirect network-level covert channels," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 546–560.

[14] M. A. Khadse and D. M. Dakhane, "A review on network covert channel construction and attack detection," *Concurrency Computation: Pract. Experience*, vol. 37, 2024, Art. no. e8316.

[15] S. J. Murdoch and S. Lewis, "Embedding covert channels into TCP/IP," in *Proc. Int. Workshop Inf. Hiding*, 2005, pp. 247–261.

[16] W. Mazurczyk, K. Powójski, and L. Caviglione, "IPV6 covert channels in the wild," in *Proc. Central Eur. CyberSecur.*, 2019, pp. 1–6.

[17] C. Heinz, W. Mazurczyk, and L. Caviglione, "Covert channels in transport layer security," in *Proc. Eur. Interdiscip. Cybersecurity*, 2020, pp. 1–6.

[18] S. Wendzel, T. Schmidbauer, S. Zillien, and J. Keller, "DYST (did you see that?): An amplified covert channel that points to previously seen data," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 1, pp. 614–631, Jan./Feb. 2025.

[19] S. Cabuk, C. E. Brodley, and C. Shields, "IP covert timing channels: Design and detection," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2004, pp. 178–187.

[20] X. Luo, E. W. Chan, P. Zhou, and R. K. Chang, "Robust network covert communications based on TCP and enumerative combinatorics," *IEEE Trans. Dependable Secure Comput.*, vol. 9, no. 6, pp. 890–902, Nov./Dec. 2012.

[21] Y. Shen, W. Yang, and L. Huang, "Concealed in web surfing: Behavior-based covert channels in HTTP," *J. Netw. Comput. Appl.*, vol. 101, pp. 83–95, 2018.

[22] Y. Li, X. Zhang, X. Xu, and Y.-A. Tan, "A robust packet-dropout covert channel over wireless networks," *IEEE Wirel. Commun.*, vol. 27, no. 3, pp. 60–65, Jun. 2020.

[23] M. B. Rosen, J. Parker, and A. J. Malozemoff, "Balboa: Bobbing and weaving around network censorship," in *Proc. USENIX Secur. Symp.*, 2021, pp. 3399–3413.

[24] J. Oakley, L. Yu, X. Zhong, G. K. Venayagamoorthy, and R. Brooks, "Protocol proxy: An FTE-based covert channel," *Comput. Secur.*, vol. 92, 2020, Art. no. 101777.

[25] D. Barradas, N. Santos, and L. Rodrigues, "DeltaShaper: Enabling unobservable censorship-resistant TCP tunneling over videoconferencing streams," in *Proc. Privacy Enhancing Technol.*, 2017, pp. 5–22.

[26] S. Li, M. Schliep, and N. Hopper, "Facet: Streaming over videoconferencing for censorship circumvention," in *Proc. Workshop Privacy Electron. Soc.*, 2014, pp. 163–172.

[27] R. McPherson, A. Houmansadr, and V. Shmatikov, "CovertCast: Using live streaming to evade internet censorship," in *Proc. Privacy Enhancing Technol.*, 2016, pp. 212–225.

[28] G. Figueira, D. Barradas, and N. Santos, "Stegozoa: Enhancing WebRTC covert channels with video steganography for internet censorship circumvention," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2022, pp. 1154–1167.

[29] B. Hahn, R. Nithyanand, P. Gill, and R. Johnson, "Games without frontiers: Investigating video games as a covert channel," in *Proc. IEEE Eur. Symp. Secur. Priv.*, 2016, pp. 63–77.

[30] P. Vines and T. Kohno, "Rook: Using video games as a low-bandwidth censorship resistant communication platform," in *Proc. Workshop Privacy Electron. Soc.*, 2015, pp. 75–84.

[31] A. Houmansadr, W. Zhou, M. Caesar, and N. Borisov, "SWEET: Serving the web by exploiting email tunnels," *IEEE/ACM Trans. Net.*, vol. 25, no. 3, pp. 1517–1527, Jun. 2017.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.

[32] Y. Chen, Y. Zhang, J. Zou, Z. Tan, Y. Huang, and N. Hu, "Stealth: A heterogeneous covert access channel for mix-net," in *Proc. IEEE Int. Conf. Data Sci. Cyberspace*, 2022, pp. 118–125.
[33] P. K. Sharma, D. Gosain, and S. Chakravarty, "Camoufler: Accessing the censored web by utilizing instant messaging channels," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2021, pp. 147–161.
[34] J.-P. Smith, L. Dolfi, P. Mittal, and A. Perrig, "QCSD: A quic client-side website-fingerprinting defence framework," in *Proc. USENIX Secur. Symp.*, 2022, pp. 771–789.
[35] H. Wang, G. Liu, Y. Dai, and J. Shi, "A detection method for cloak covert channel based on distribution of TCP burst size," *J. Inf. Hiding Multim. Signal Process.*, vol. 6, no. 4, pp. 750–759, 2015.
[36] H. Li, T. Song, and Y. Yang, "Generic and sensitive anomaly detection of network covert timing channels," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 5, pp. 4085–4100, Sep./Oct. 2023.
[37] M. Shen et al., "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Commun. Surv. Tut.*, vol. 25, no. 1, pp. 791–824, First Quarter 2023.
[38] D. Barradas, N. Santos, and L. Rodrigues, "Effective detection of multimedia protocol tunneling using machine learning," in *Proc. USENIX Secur. Symp.*, 2018, pp. 169–185.
[39] W. Jia, "Quicstep: Circumventing QUIC-based censorship," Master's thesis, Dept. Comput. Sci., Princeton Univ., 2023.
[40] B. Gao, W. Liu, G. Liu, F. Nie, and J. Huang, "Multi-level resource-coherented graph learning for website fingerprinting attacks," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 693–708, 2025.
[41] D. Herrmann, R. Wendolsky, and H. Federrath, "Website fingerprinting: Attacking popular privacy enhancing technologies with the multinomial naïve-bayes classifier," in *Proc. ACM Workshop Cloud Comput. Secur.*, 2009, pp. 31–42.
[42] A. Panchenko, L. Niessen, A. Zinnen, and T. Engel, "Website fingerprinting in onion routing based anonymization networks," in *Proc. ACM Workshop Privacy Electron. Soc.*, 2011, pp. 103–114.
[43] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 1171–1179.
[44] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. Conf. Comput. Commun. Secur.*, 2018, pp. 1928–1943.
[45] X. Xiao, W. Xiao, R. Li, X. Luo, H. Zheng, and S. Xia, "EBSNN: Extended byte segment neural network for network traffic classification," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3521–3538, Sep./Oct. 2022.
[46] X. Yun, Y. Wang, Y. Zhang, C. Zhao, and Z. Zhao, "Encrypted TLS traffic classification on cloud platforms," *IEEE/ACM Trans. Net.*, vol. 31, no. 1, pp. 164–177, Feb. 2023.
[47] J. Xing, Q. Kang, and A. Chen, "Netwarden: Mitigating network covert channels while preserving performance," in *Proc. USENIX Secur. Symp.*, 2020, pp. 2039–2056.
[48] S. Zillien and S. Wendzel, "Weaknesses of popular and recent covert channel detection methods and a remedy," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 5156–5167, Nov./Dec. 2023.
[49] G. Zhou, Z. Liu, C. Fu, Q. Li, and K. Xu, "An efficient design of intelligent network data plane," in *Proc. USENIX Secur. Symp.*, 2023, pp. 6203–6220.
[50] D. Barradas, N. Santos, L. Rodrigues, S. Signorello, F. M. Ramos, and A. Madeira, "Flowlens: Enabling efficient flow classification for ml-based network security applications," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021, pp. 1–18.
[51] M. Shen, M. Wei, L. Zhu, and M. Wang, "Classification of encrypted traffic with second-order markov chains and application attribute bigrams," *IEEE Trans. Inf. Forensics Secur.*, vol. 12, no. 8, pp. 1830–1843, Aug. 2017.
[52] W. Li et al., "Prism: Real-time privacy protection against temporal network traffic analyzers," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 2524–2537, 2023.
[53] T. Mikolov, M. Karafiát, L. Burget, J. Cernockỳ, and S. Khudanpur, "Recurrent neural network based language model," in *Proc. Interspeech*, 2010, pp. 1045–1048.
[54] Y. Wang, H. Xu, Z. Guo, Z. Qin, and K. Ren, "snWF: Website fingerprinting attack by ensembling the snapshot of deep learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1214–1226, 2022.
[55] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009.
[56] Weasel: A stealthy DNS beacon. Accessed: Feb 18, 2025. [Online]. Available: https://github.com/facebookarchive/WEASEL
[57] IPV6Teal: Stealthy data exfiltration via IPV6 covert channel. Accessed: Feb 18, 2025. [Online]. Available: https://github.com/christophetd/IPv6teal



Jianan Huang received the BE degree from Electronic and Information Engineering in Zhejiang Agriculture and Forestry University in 2021. He is currently working toward the PhD degree with the Nanjing University of Science and Technology. His research interests include network covert channels and network traffic analysis.



Weiwei Liu (Member, IEEE) received the BS degree in automation and the PhD degree in control science and engineering from the Nanjing University of Science and Technology in 2010 and 2015, respectively. From 2014 to 2015, he was a visiting scholar with the Department of Computer Science, University of California at Davis. He is currently an associate professor with the Nanjing University of Science and Technology. His research interests include network covert communication and encrypted traffic analysis.



Guangjie Liu (Member, IEEE) received the BS degree in electrical and computer engineering and the PhD degree in control science and engineering from the Nanjing University of Science and Technology in 2002 and 2007, respectively. From 2016 to 2017, he was a visiting scholar with the Department of Computer Science, University of California at Davis. He is currently a professor with the Nanjing University of Information Science and Technology. His research interests are network and information security.



Bo Gao (Member, IEEE) received the BS degree in physics from Jilin University, Jilin, in 2016, and the MEng degree in control engineering from the Nanjing University of Science and Technology, Nanjing, China, in 2020. He is currently working toward the PhD degree with the Nanjing University of Science and Technology. His research interests include anonymity networks and network traffic analysis.



Fengyuan Nie received the BS degree in automation from the Shanghai University of Electric Power in July 2020. He is currently working toward the PhD degree with the Nanjing University of Science and Technology. His research interests include Internet of Things, network traffic classification and deep learning.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 08:14:13 UTC from IEEE Xplore. Restrictions apply.